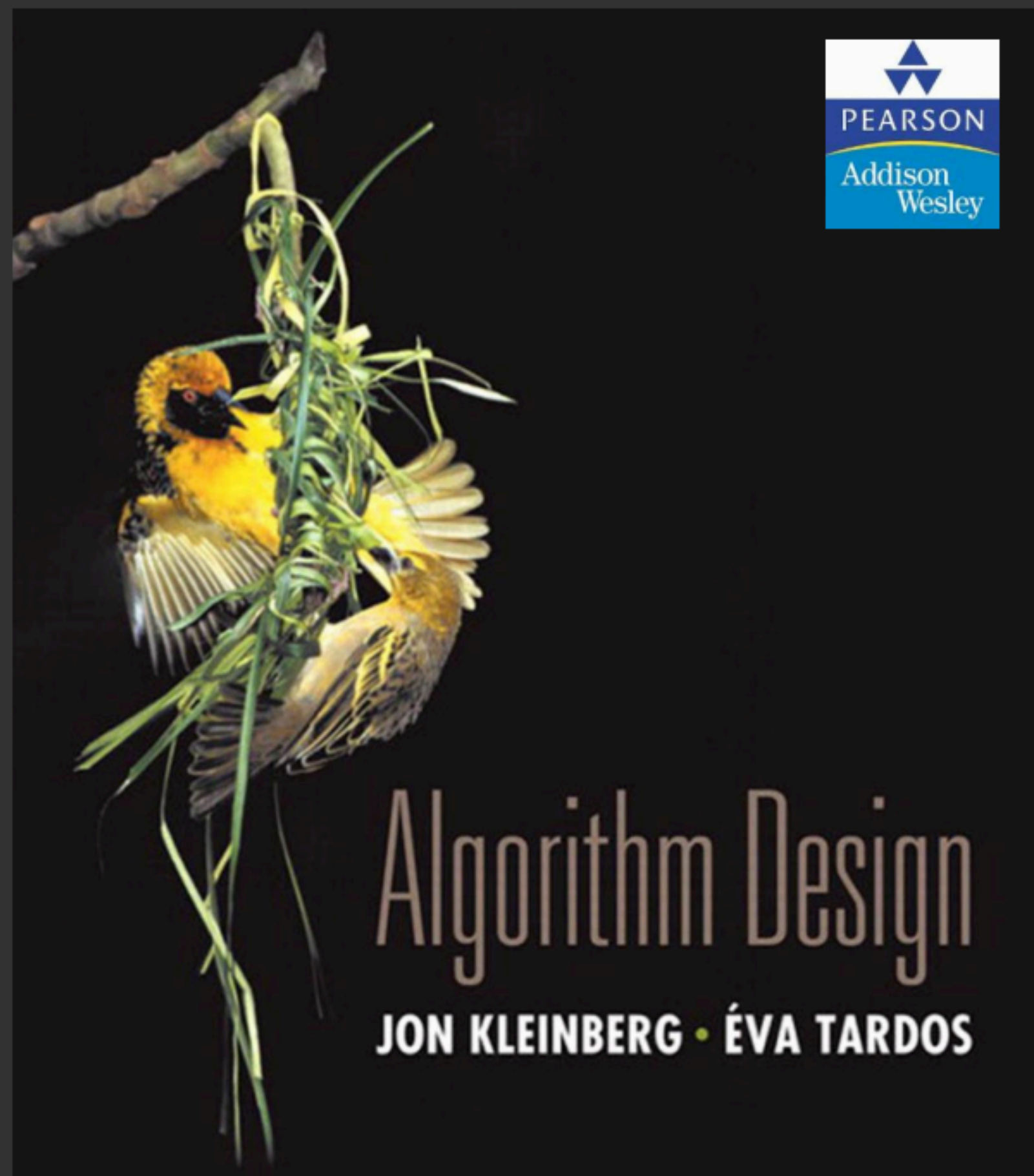




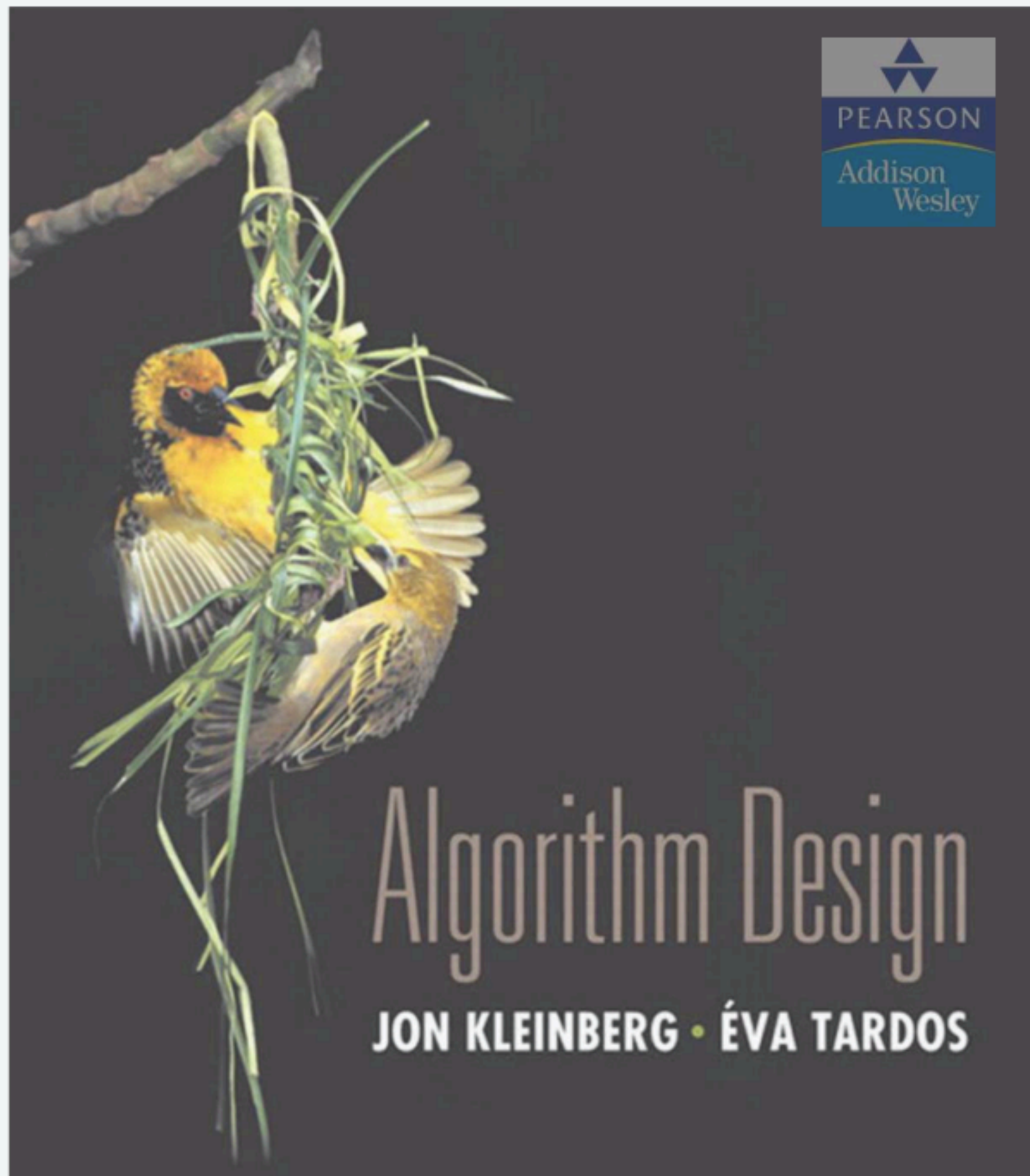
6. DYNAMIC PROGRAMMING II

- ▶ *sequence alignment*
- ▶ *Hirschberg's algorithm*
- ▶ *Bellman–Ford–Moore algorithm*

Lecture slides by Kevin Wayne

Copyright © 2005 Pearson-Addison Wesley

<http://www.cs.princeton.edu/~wayne/kleinberg-tardos>



SECTION 6.6

6. DYNAMIC PROGRAMMING II

- ▶ *sequence alignment*
- ▶ *Hirschberg's algorithm*
- ▶ *Bellman–Ford–Moore algorithm*

String similarity

Q. How similar are two strings?

Ex. occurance and occurrence.

o	c	u	r	r	a	n	c	e	-
o	c	c	u	r	r	e	n	c	e

6 mismatches, 1 gap

↓ CERCO DI ALLINEARE

o	c	-	u	r	r	-	a	n	c	e
o	c	c	u	r	r	e	-	n	c	e

0 mismatches, 3 gaps

o	c	-	u	r	r	a	n	c	e
o	c	c	u	r	r	e	n	c	e

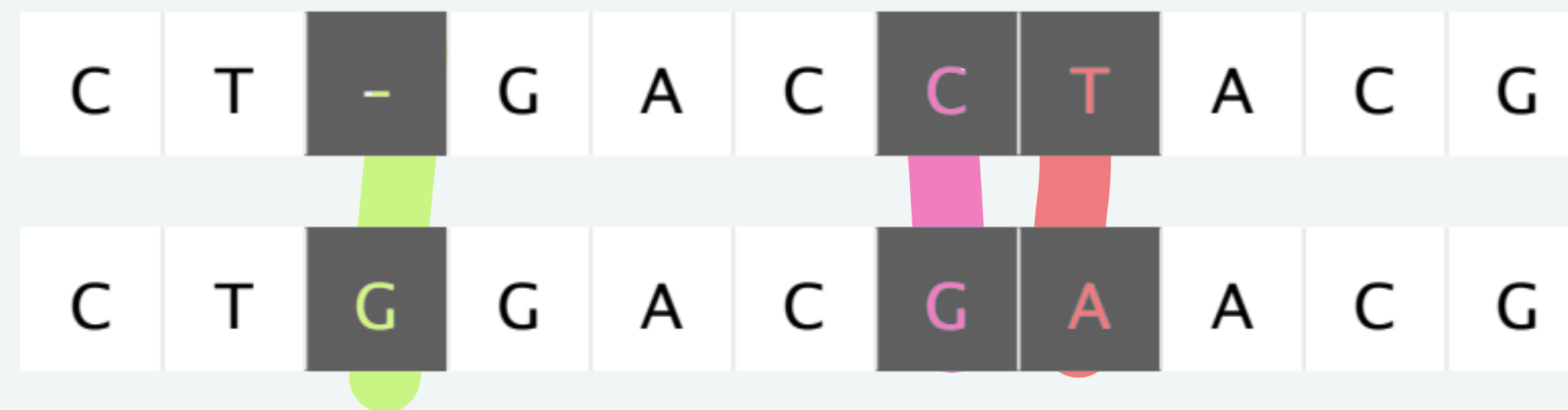
1 mismatch, 1 gap

↗ UN'ALTRO

Edit distance $\leftarrow$ PREVIOUS | GAP & MISMATCH MINOR!

Edit distance. [Levenshtein 1966, Needleman-Wunsch 1970]

- Gap penalty δ ; mismatch penalty α_{pq} .
- Cost = sum of gap and mismatch penalties.



$$\text{cost} = \delta + \alpha_{CG} + \alpha_{TA}$$

$\uparrow$ ALIGNMENT COST
 $\uparrow$ COST TO GAP (NEH0)LO1
 $\uparrow$ COST TO MISMATCH (NEH0)LO2

QUESTI NON SONO MISMATCH
 assuming $\alpha_{AA} = \alpha_{CC} = \alpha_{GG} = \alpha_{TT} = 0$

Applications. Bioinformatics, spell correction, machine translation, speech recognition, information extraction, ...

Confusion matrix for English text

Number of times one letter was substituted for another.

COSTO DI MISSPELLING

Come leggere la tabella:

- L'asse X (orizzontale) rappresenta la lettera scritta male (cioè quella sbagliata, che è stata usata per errore).
- L'asse Y (verticale) rappresenta la lettera corretta (cioè quella che avrebbe dovuto essere scritta).
- Le celle contengono il numero di volte in cui una lettera è stata sostituita per un'altra. Più il numero è alto, più frequentemente quell'errore è avvenuto.

X	Y (correct)																									
	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	0	0	7	1	342	0	0	2	118	0	1	0	0	3	76	0	0	1	35	9	9	0	1	0	5	0
b	0	0	9	9	2	2	3	1	0	0	0	5	11	5	0	10	0	0	2	1	0	0	8	0	0	0
c	6	5	0	16	0	9	5	0	0	0	1	0	7	9	1	10	2	5	39	40	1	3	7	1	1	0
d	1	10	13	0	12	0	5	5	0	0	2	3	7	3	0	1	0	43	30	22	0	0	4	0	2	0
e	388	0	3	11	0	2	2	0	89	0	0	3	0	5	93	0	0	14	12	6	15	0	1	0	18	0
f	0	15	0	3	1	0	5	2	0	0	0	3	4	1	0	0	0	6	4	12	0	0	2	0	0	0
g	4	1	11	11	9	2	0	0	0	1	1	3	0	0	2	1	3	5	13	21	0	0	1	0	3	0
h	1	8	0	3	0	0	0	0	0	0	2	0	12	14	2	3	0	3	1	11	0	0	2	0	0	0
i	103	0	0	0	146	0	1	0	0	0	0	6	0	0	49	0	0	0	2	1	47	0	2	1	15	0
j	0	1	1	9	0	0	1	0	0	0	0	2	1	0	0	0	0	0	5	0	0	0	0	0	0	0
k	1	2	8	4	1	1	2	5	0	0	0	0	5	0	2	0	0	0	6	0	0	0	4	0	0	3
l	2	10	1	4	0	4	5	6	13	0	1	0	0	14	2	5	0	11	10	2	0	0	0	0	0	0
m	1	3	7	8	0	2	0	6	0	0	4	4	0	180	0	6	0	0	9	15	13	3	2	2	3	0
n	2	7	6	5	3	0	1	19	1	0	4	35	78	0	0	7	0	28	5	7	0	0	1	2	0	2
o	91	1	1	3	116	0	0	0	25	0	2	0	0	0	0	14	0	2	4	14	39	0	0	0	18	0
p	0	11	1	2	0	6	5	0	2	9	0	2	7	6	15	0	0	1	3	6	0	4	1	0	0	0
q	0	0	1	0	0	0	27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
r	0	14	0	30	12	2	2	8	2	0	5	8	4	20	1	14	0	0	12	22	4	0	0	1	0	0
s	11	8	27	33	35	4	0	1	0	1	0	27	0	6	1	7	0	14	0	15	0	0	5	3	20	1
t	3	4	9	42	7	5	19	5	0	1	0	14	9	5	5	6	0	11	37	0	0	2	19	0	7	6
u	20	0	0	0	44	0	0	0	64	0	0	0	0	2	43	0	0	4	0	0	0	0	2	0	8	0
v	0	0	7	0	0	3	0	0	0	0	0	1	0	0	1	0	0	0	8	3	0	0	0	0	0	0
w	2	2	1	0	1	0	0	2	0	0	1	0	0	0	0	7	0	6	3	3	1	0	0	0	0	0
x	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	0	0	0	0	0	0	0
y	0	0	2	0	15	0	1	7	15	0	0	0	2	0	6	1	0	7	36	8	5	0	0	1	0	0
z	0	0	0	7	0	0	0	0	0	0	0	7	5	0	0	0	0	2	21	3	0	0	0	0	3	0

BLOSUM matrix for proteins

BLOcks SUBstitution Matrix: is a substitution matrix used for sequence alignment of proteins.

BLOSUM matrices are used to score alignments between evolutionarily divergent protein sequences.

POSITIVO = BUONO

	A	R	N	D	C	Q	E	G	H	I	L	K	M	F	P	S	T	W	Y	V
A	7	-3	-3	-3	-1	-2	-2	0	-3	-3	-3	-1	-2	-4	-1	2	0	-5	-4	-1
R	-3	9	-1	-3	-6	1	-1	-4	0	-5	-4	3	-3	-5	-3	-2	-2	-5	-4	-4
N	-3	-1	9	2	-5	0	-1	-1	1	-6	-6	0	-4	-6	-4	1	0	-7	-4	-5
D	-3	-3	2	10	-7	-1	2	-3	-2	-7	-7	-2	-6	-6	-3	-1	-2	-8	-6	-6
C	-1	-6	-5	-7	13	-5	-7	-6	-7	-2	-3	-6	-3	-4	-6	-2	-2	-5	-5	-2
Q	-2	1	0	-1	-5	9	3	-4	1	-5	-4	2	-1	-5	-3	-1	-1	-4	-3	-4
E	-2	-1	-1	2	-7	3	8	-4	0	-6	-6	1	-4	-6	-2	-1	-2	-6	-5	-4
G	0	-4	-1	-3	-6	-4	-4	9	-4	-7	-7	-3	-5	-6	-5	-1	-3	-6	-6	-6
H	-3	0	1	-2	-7	1	0	-4	12	-6	-5	-1	-4	-2	-4	-2	-3	-4	3	-5
I	-3	-5	-6	-7	-2	-5	-6	-7	-6	7	2	-5	2	-1	-5	-4	-2	-5	-3	4
L	-3	-4	-6	-7	-3	-4	-6	-7	-5	2	6	-4	3	0	-5	-4	-3	-4	-2	1
K	-1	3	0	-2	-6	2	1	-3	-1	-5	-4	8	-3	-5	-2	-1	-1	-6	-4	-4
M	-2	-3	-4	-6	-3	-1	-4	-5	-4	2	3	-3	9	0	-4	-3	-1	-3	-3	1
F	-4	-5	-6	-6	-4	-5	-6	-6	-2	-1	0	-5	0	10	-6	-4	-4	0	4	-2
P	-1	-3	-4	-3	-6	-3	-2	-5	-4	-5	-5	-2	-4	-6	12	-2	-3	-7	-6	-4
S	2	-2	1	-1	-2	-1	-1	-1	-2	-4	-4	-1	-3	-4	-2	7	2	-6	-3	-3
T	0	-2	0	-2	-2	-1	-2	-3	-3	-2	-3	-1	-1	-4	-3	2	8	-5	-3	0
W	-5	-5	-7	-8	-5	-4	-6	-6	-4	-5	-4	-6	-3	0	-7	-6	-5	16	3	-5
Y	-4	-4	-4	-6	-5	-3	-5	-6	3	-3	-2	-4	-3	4	-6	-3	-3	3	11	-3
V	-1	-4	-5	-6	-2	-4	-4	-6	-5	4	1	-4	1	-2	-4	-3	0	-5	-3	7

Edit distance

One more example.

What is edit distance between these two strings?

P A L E T T E

P A L A T E

Assume gap penalty = 2 and mismatch penalty = 1.

QUANTO COSTANO → QUINDI MEGLIO
FARE + MISMATCH
CHE PUNISCONO MENO

P	A	L	E	T	T	E
P	A	L	-	A	T	E

1 gap, 1 mismatch

Sequence alignment

Goal. Given two strings $x_1 x_2 \dots x_m$ and $y_1 y_2 \dots y_n$, find a min-cost alignment.

Def. An **alignment** M is a set of ordered pairs $x_i - y_j$ such that each character appears in at most one pair and no crossings.

Def. The **cost** of an alignment M is:

$$\text{cost}(M) = \underbrace{\sum_{(x_i, y_j) \in M} \alpha_{x_i y_j}}_{\text{mismatch}} + \underbrace{\sum_{i: x_i \text{ unmatched}} \delta}_{\text{gap PER STRINGA X}} + \underbrace{\sum_{j: y_j \text{ unmatched}} \delta}_{\text{gap PER Y}}$$

SOMMA MISMATCH

$x_i - y_j$ and $x_{i'} - y_{j'}$ cross if $i < i'$, but $j > j'$

DUE COPIE DI INCROCI

IN CROCIAMO

PER $x_i - y_5$
intende:

CARATTERE I STRINGA X
CON J STRINGA Y

X: ¹A ²B ³C ⁴D

Y: ¹A ²C ³B ⁴D

$x_1 - y_3$

--A B C D
A C B D--
3

$\tilde{x} \tilde{y} \tilde{x}'$ (ANCHE IL CONTRARIO)

$\tilde{x} \tilde{y} \tilde{x}'$ (CROSS)

<u>x_1</u>	x_2	x_3	x_4	x_5	x_6
C	T	A	C	C	-
-	T	A	C	A	T
y_1	y_2	y_3	y_4	<u>y_5</u>	y_6

an alignment of CTACCG and TACATG

$M = \{ x_2 - y_1, x_3 - y_2, x_4 - y_3, x_5 - y_4, x_6 - y_6 \}$

INSIEME DI COPPIE

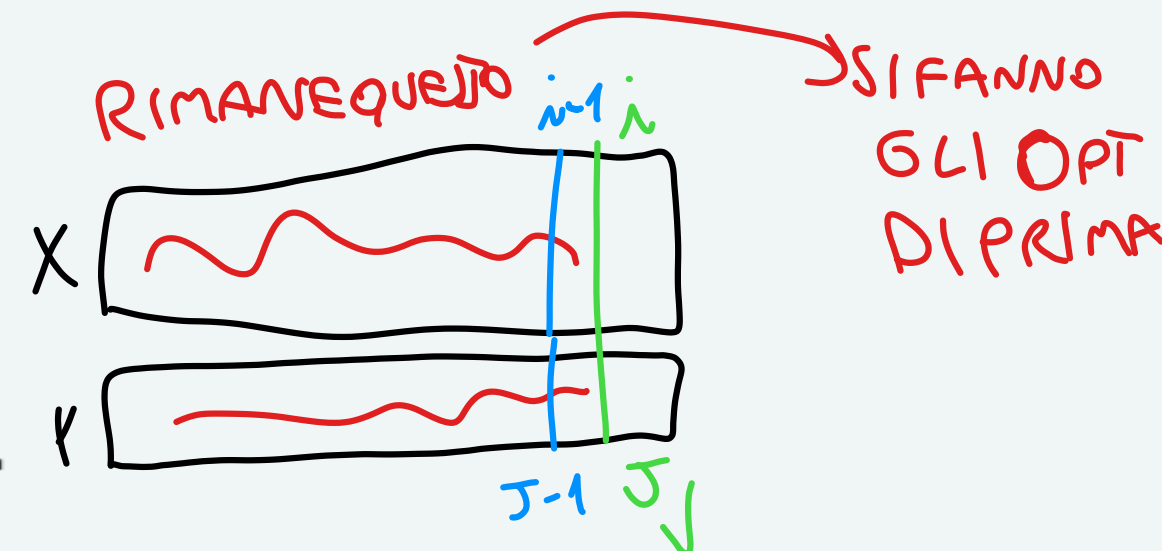
DA UN CARATTERE DI
X CON Y

NO INCROCI
UN CARATTERE PUO' ESSERE MAX IN 1 COPPIA

Sequence alignment: problem structure

Def. $OPT(i, j)$ = min cost of aligning prefix strings $x_1 x_2 \dots x_i$ and $y_1 y_2 \dots y_j$.

Goal. $OPT(m, n)$.

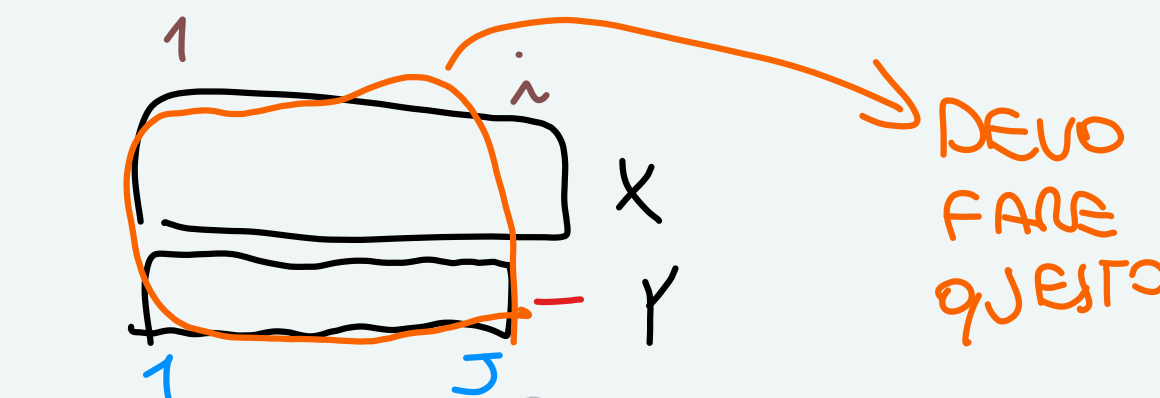


Case 1. $OPT(i, j)$ matches $x_i - y_j$.

Pay mismatch for $x_i - y_j$ + min cost of aligning $x_1 x_2 \dots x_{i-1}$ and $y_1 y_2 \dots y_{j-1}$.

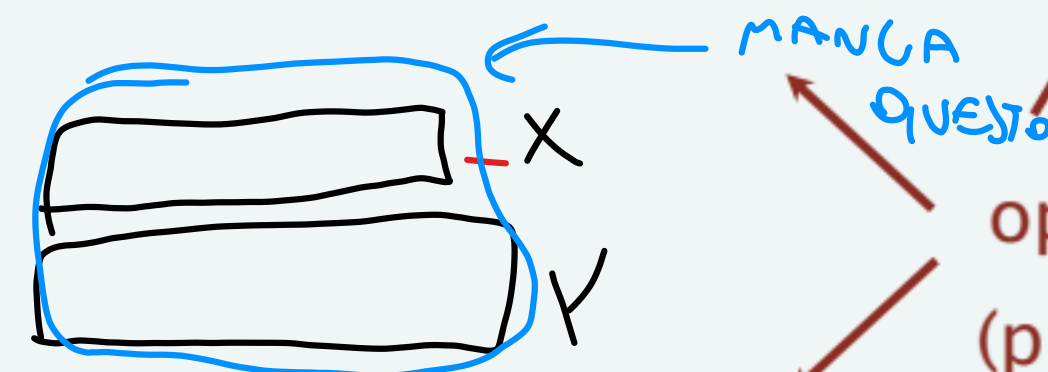
Case 2a. $OPT(i, j)$ leaves x_i unmatched.

Pay gap for x_i + min cost of aligning $x_1 x_2 \dots x_{i-1}$ and $y_1 y_2 \dots y_j$.



Case 2b. $OPT(i, j)$ leaves y_j unmatched.

Pay gap for y_j + min cost of aligning $x_1 x_2 \dots x_i$ and $y_1 y_2 \dots y_{j-1}$.



optimal substructure property
(proof via exchange argument)

Bellman equation.

IL BOSS

$$OPT(i, j) = \begin{cases} j\delta & \text{if } i = 0 \\ i\delta & \text{if } j = 0 \\ \min \begin{cases} \alpha_{x_i y_j} + OPT(i-1, j-1) & \text{MISMATCH} \\ \delta + OPT(i-1, j) & \text{GAP} \\ \delta + OPT(i, j-1) & \text{GAP} \end{cases} & \text{otherwise} \end{cases}$$

RICORS.

Sequence alignment: bottom-up algorithm

SEQUENCE-ALIGNMENT($m, n, x_1, \dots, x_m, y_1, \dots, y_n, \delta, \alpha$)

FOR $i = 0$ TO m

$M[i, 0] \leftarrow i\delta.$

FOR $j = 0$ TO n

$M[0, j] \leftarrow j\delta.$

FOR $i = 1$ TO m

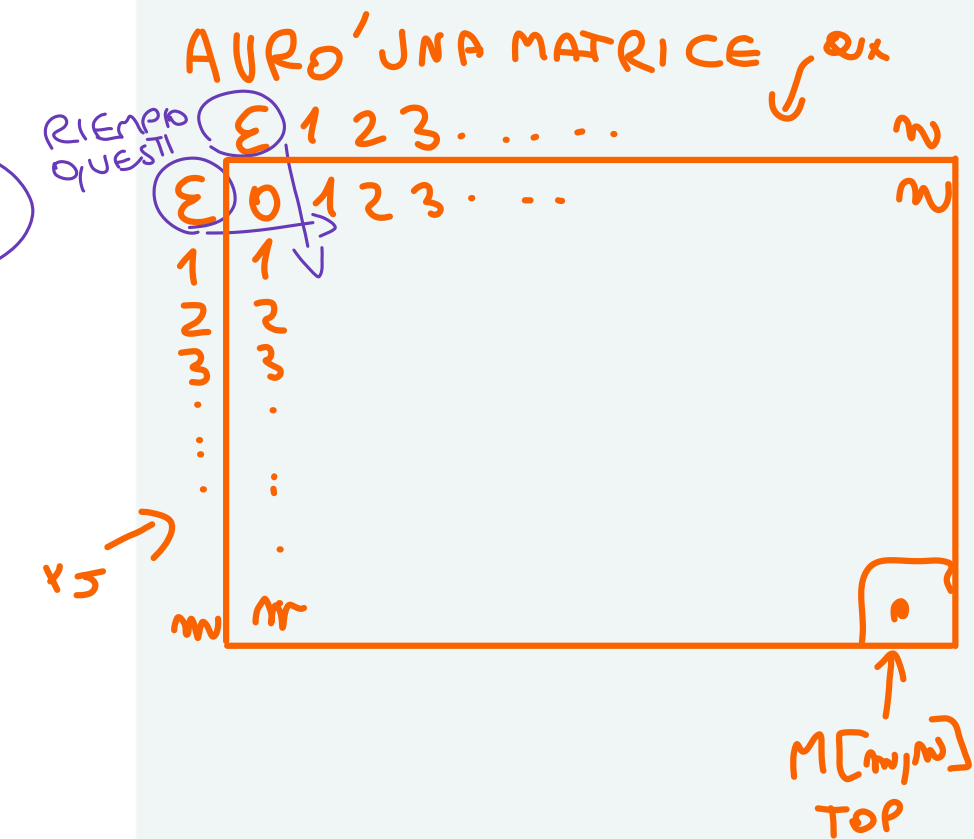
FOR $j = 1$ TO n

$M[i, j] \leftarrow \min \{ \alpha_{x_i y_j} + M[i-1, j-1],$

$\delta + M[i-1, j],$

$\delta + M[i, j-1] \}.$

RETURN $M[m, n].$



already
computed

Sequence alignment: traceback

	P	A	L	A	T	E
P	0	2	4	6	8	10
A	2	0	2	4	6	8
L	4	2	0	2	4	6
E	6	4	2	0	2	4
T	8	6	4	2	1	3
T	10	8	6	4	3	1
E	12	10	8	6	5	3
E	14	12	10	8	7	5

SALE RICOSTRUISCE

P	A	L	E	T	T	E
P	A	L	-	A	T	E

1 gap, 1 mismatch
(gap penalty = 2, mismatch penalty = 1)

Backtracking (ricostruzione):
Parti da $M[m][n]$ (in basso a destra) e risalisci fino a $M[0][0]$.
A ogni passo scegli la direzione ($\nwarrow$ $\uparrow$ $\swarrow$) che ha prodotto il costo minimo.
Segui solo celle compatibili col valore corrente.
Quando arrivi in alto a sinistra, hai ricostruito l'allineamento (con match, mismatch, e gap).

POTEVOSCEGLIERE

Sequence alignment: analysis

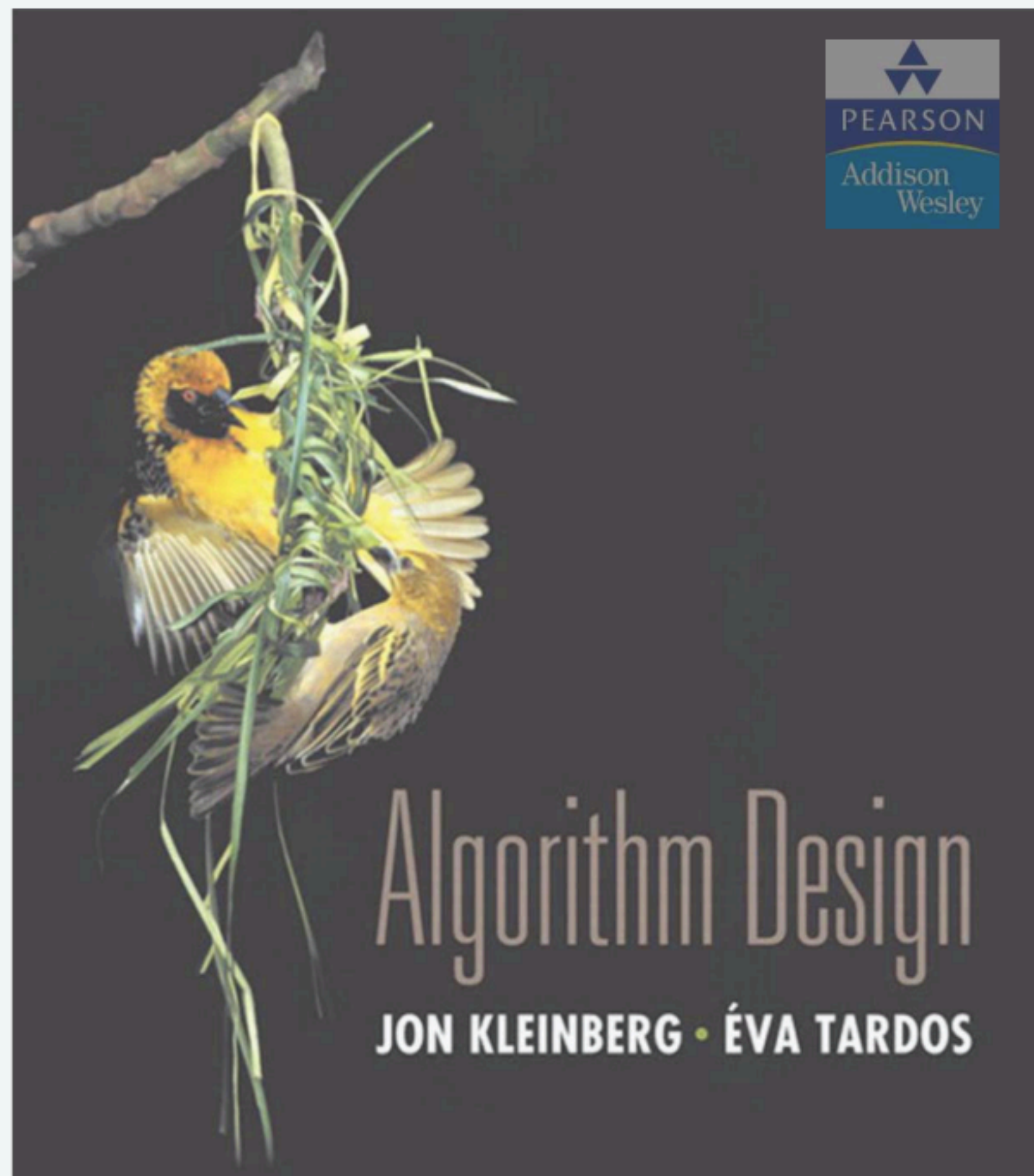
Theorem. The DP algorithm computes the edit distance (and an optimal alignment) of two strings of lengths m and n in $\Theta(mn)$ time and space.

Pf.

- Algorithm computes edit distance.
- Can trace back to extract optimal alignment itself. ▪

Can we improve the space used by the algorithm?

COSTO SPAZIALE
CON MATRICE
ELEVATO



SECTION 6.7

6. DYNAMIC PROGRAMMING II

- ▶ *sequence alignment*
- ▶ *Hirschberg's algorithm*
- ▶ *Bellman–Ford–Moore algorithm*

Sequence alignment in linear space

ALGORITMO CHE MIGLIAORA QUANTO DETTO PRIMA

Theorem. [Hirschberg] There exists an algorithm to find an optimal alignment in $O(mn)$ time and $O(m + n)$ space.

- Clever combination of divide-and-conquer and dynamic programming.

Programming
Techniques

G. Manacher
Editor

A Linear Space Algorithm for Computing Maximal Common Subsequences

D.S. Hirschberg
Princeton University

The problem of finding a longest common subsequence of two strings has been solved in quadratic time and space. An algorithm is presented which will solve this problem in quadratic time and in linear space.

Key Words and Phrases: subsequence, longest common subsequence, string correction, editing

CR Categories: 3.63, 3.73, 3.79, 4.22, 5.25

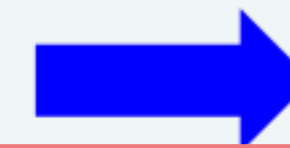


ROSARIO
GUGLIOTTA

Improving the space: first trick

		P	A	L	A	T	E	
		0	2	4	6	8	10	12
P		2	0	2	4	6	8	10
A		4	2	0	2	4	6	8
L		6	4	2	0	2	4	6
E		8	6	4	2	1	3	4
T		10	8	6	4	3	1	3
T		12	10	8	6	5	3	2
E		14	12	10	8	7	5	3

To compute the next column/row of the matrix we need only the previous column/row



maintain only 2 column/row a time



$O(m+n)$ space

⚠ COME FACCIAMO
A RICOSTRUIRE

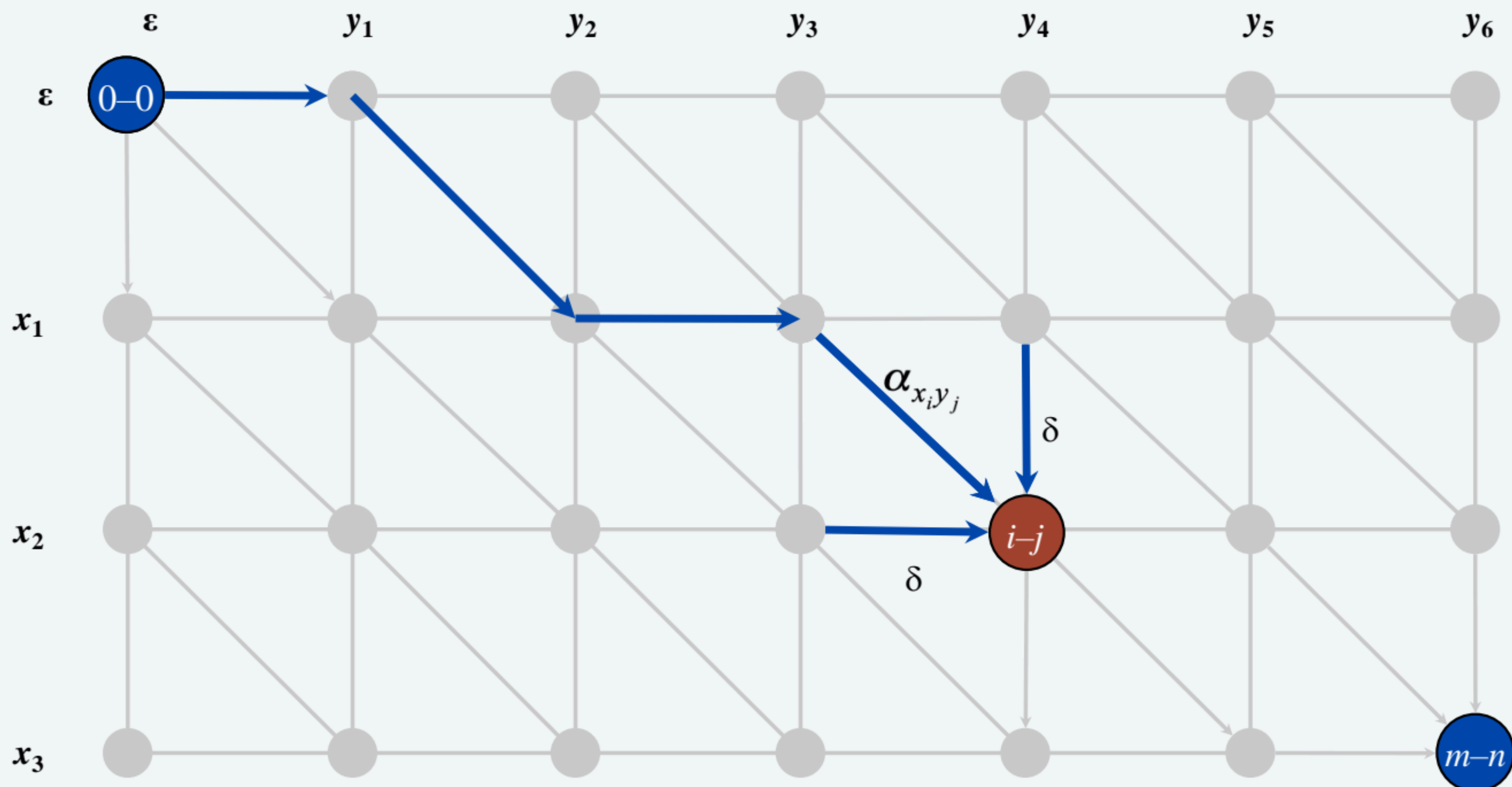
SE HO CHE CAZZO CI FACCIAMO CON

notice: this allows to compute the edit distance but not the alignment

Hirschberg's algorithm

Edit distance graph.

- Let $f(i, j)$ denote length of shortest path from $(0,0)$ to (i, j) .
- Lemma: $f(i, j) = OPT(i, j)$ for all i and j .



Hirschberg's algorithm

Edit distance graph.

- Let $f(i, j)$ denote length of shortest path from $(0,0)$ to (i, j) .
- Lemma: $f(i, j) = OPT(i, j)$ for all i and j .

VO GLIAMO
DIMOSTRARE NUOVO ALG = VECCHIO

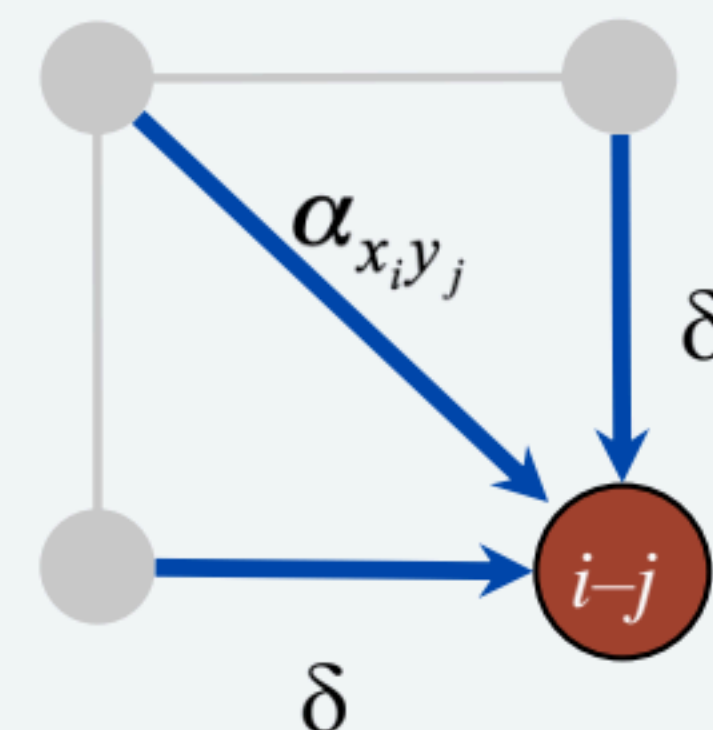
Pf of Lemma. [by strong induction on $i + j$]

- Base case: $f(0, 0) = OPT(0, 0) = 0$. CASO 0
- Inductive hypothesis: assume true for all (i', j') with $i' + j' < i + j$.
- Last edge on shortest path to (i, j) is from $(i-1, j-1)$, $(i-1, j)$, or $(i, j-1)$.
CASO 1 CASO 2 CASO 3
- Thus,

$$\begin{aligned} f(i, j) &= \min\{\alpha_{x_i y_j} + f(i-1, j-1), \delta + f(i-1, j), \delta + f(i, j-1)\} \\ &= \min\{\alpha_{x_i y_j} + OPT(i-1, j-1), \delta + OPT(i-1, j), \delta + OPT(i, j-1)\} \\ &= OPT(i, j) \end{aligned}$$

inductive
hypothesis

Bellman
equation

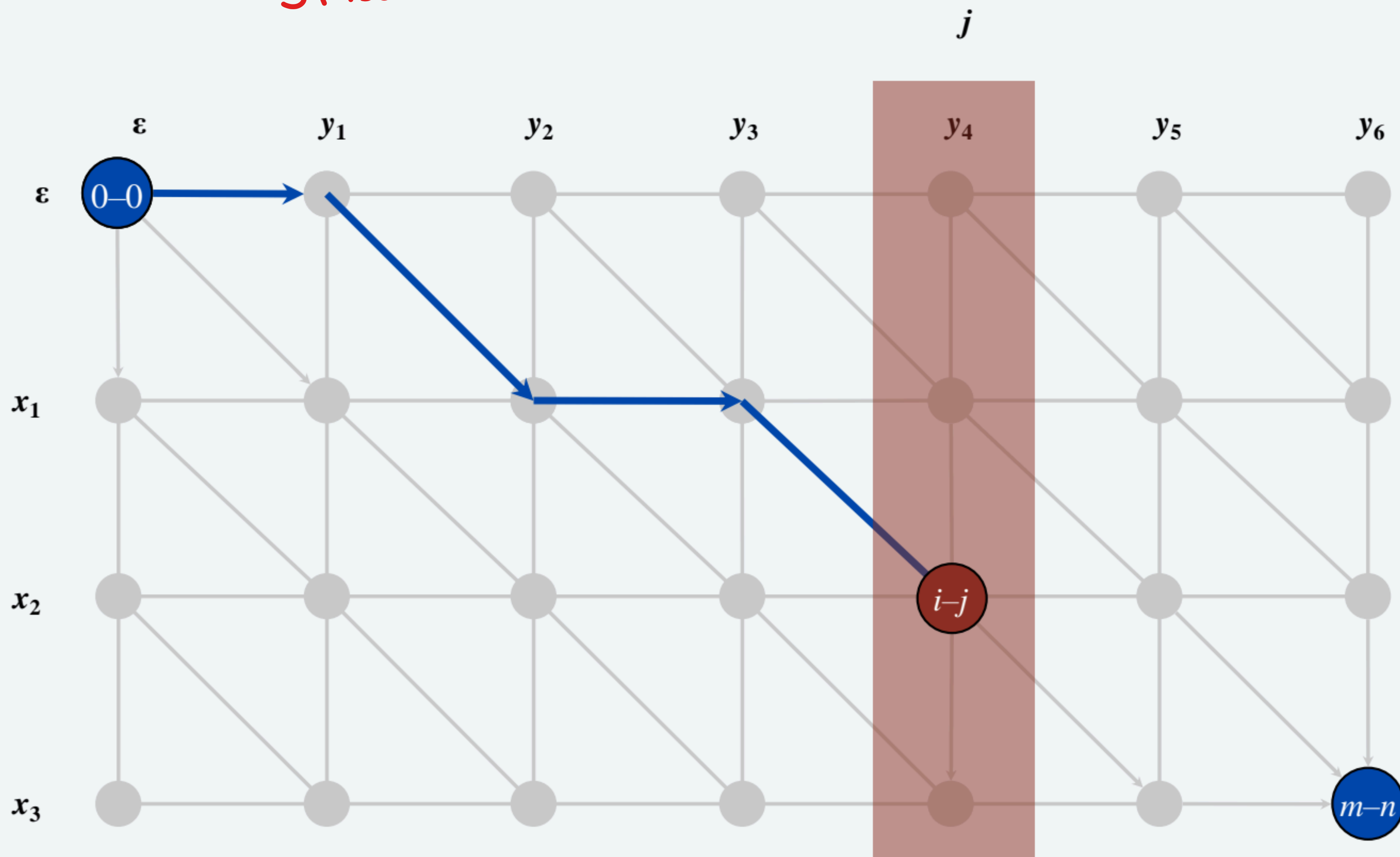


Hirschberg's algorithm

Edit distance graph.

- Let $f(i, j)$ denote length of shortest path from $(0,0)$ to (i, j) .
- Lemma: $f(i, j) = OPT(i, j)$ for all i and j .
- Can compute $f(\cdot, j)$ for any j in $O(mn)$ time and $O(m + n)$ space.

*si con
j fisso*



5. In sintesi

Questa slide spiega che:

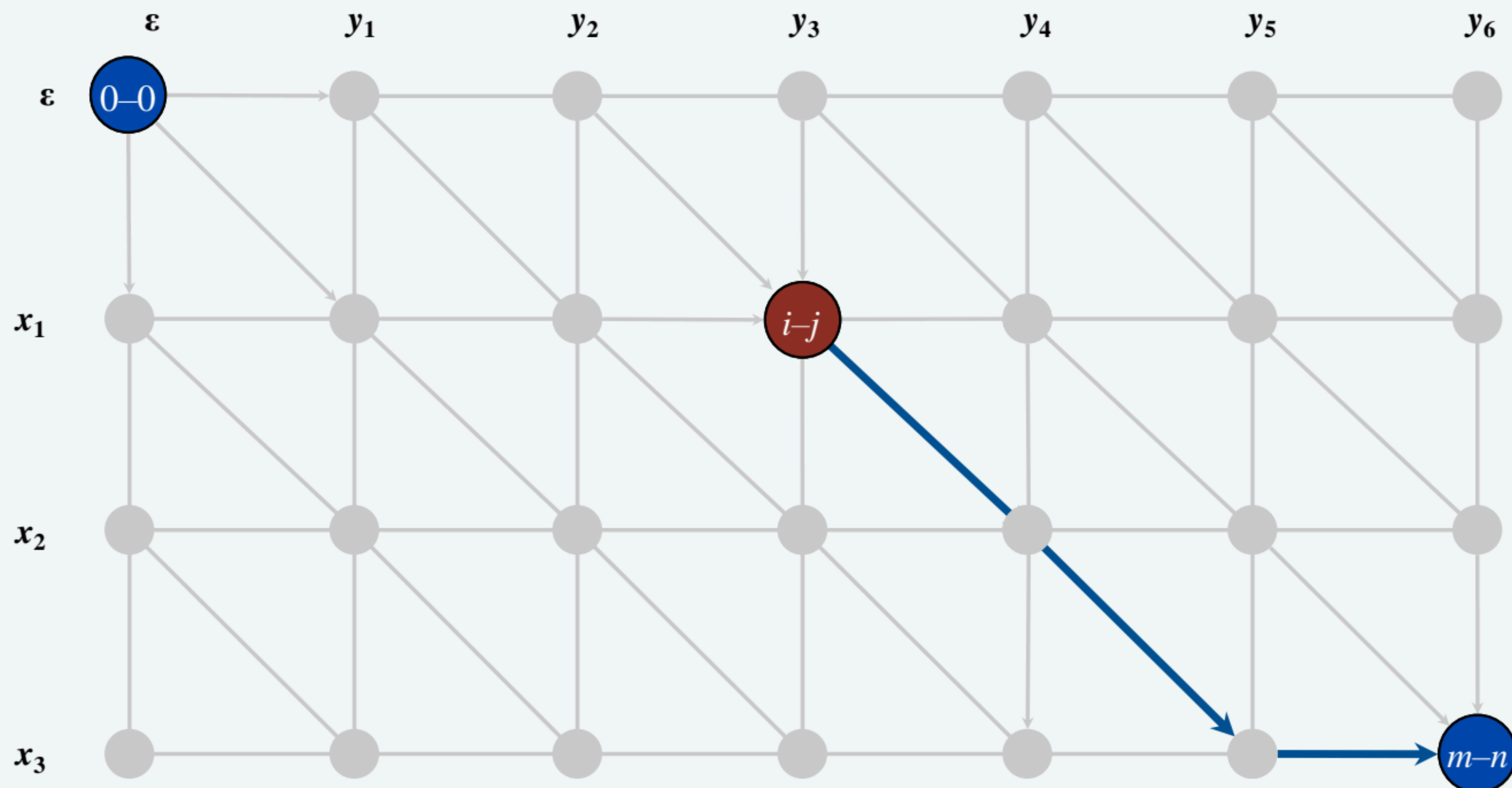
- possiamo calcolare il costo minimo dell'allineamento fino a una colonna qualsiasi j
- usando poco spazio ($O(m + n)$) invece di memorizzare tutta la matrice
- ed è la base dell'ottimizzazione dell'algoritmo di Hirschberg

Hirschberg's algorithm

Edit distance graph.

- Let $g(i, j)$ denote length of shortest path from (i, j) to (m, n) .

V E D I S L I D E J U C C

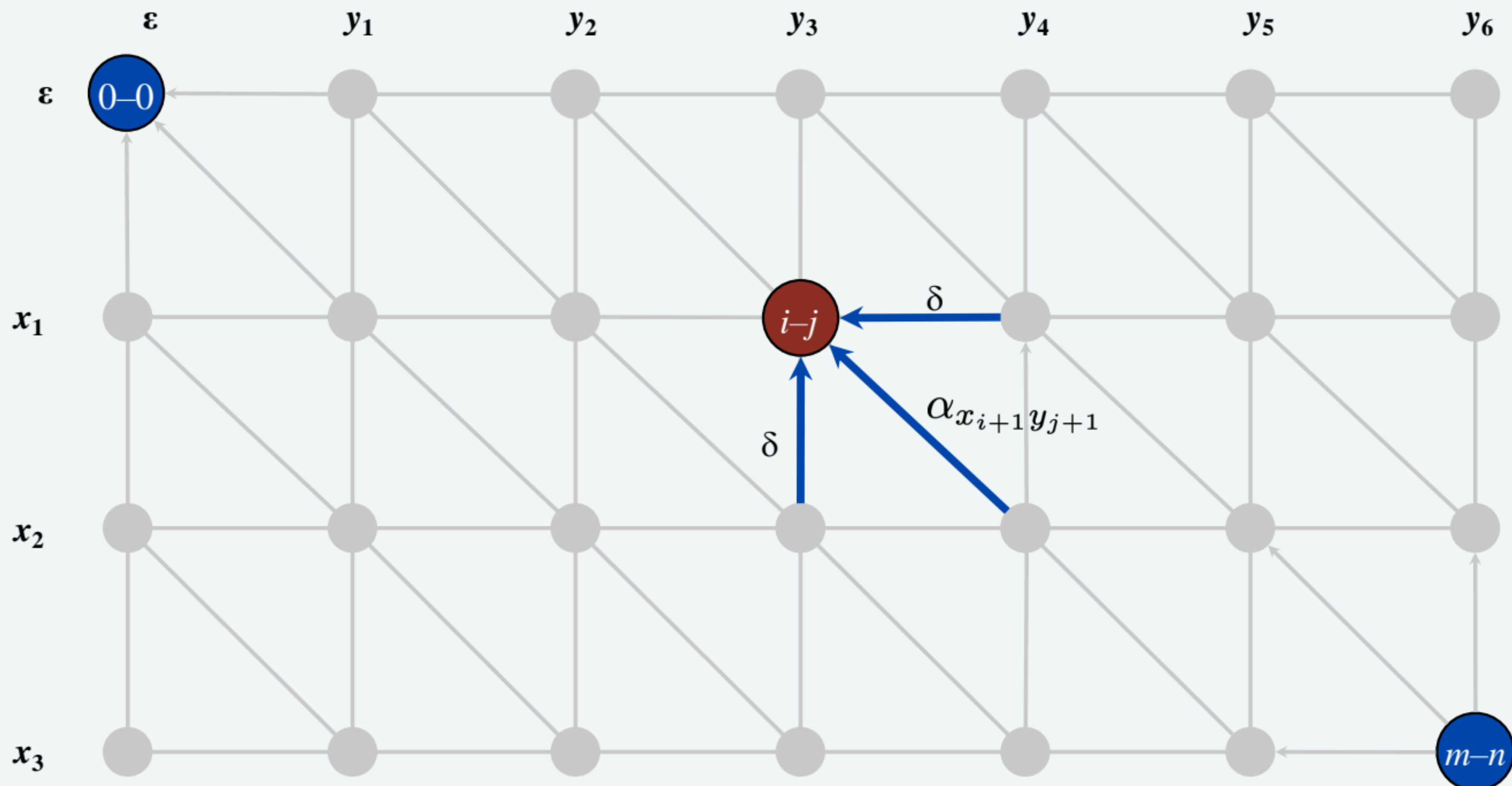


Hirschberg's algorithm

Edit distance graph.

- Let $g(i, j)$ denote length of shortest path from (i, j) to (m, n) .
- Can compute $g(i, j)$ by reversing the edge orientations and inverting the roles of $(0, 0)$ and (m, n) .

Andare avanti da ogni (i, j) richiederebbe calcolare e ricordare separatamente ogni cammino da lì a (m, n) . Invertendo, fai tutto in una passata, senza moltiplicare spazio né tempo.



L'obiettivo è calcolare un allineamento ottimo usando spazio ridotto $O(n)$.

Fase 1 - Calcolo da $(0,0)$ a colonna j

Si sceglie una colonna j (tipicamente metà di y) e si calcolano i costi $f(i, j)$ per arrivare da $(0,0)$ a ogni cella della colonna j . Questo si fa con due righe alla volta, dimenticando quelle precedenti.

Problema

A questo punto non sappiamo ancora quale riga i della colonna j è attraversata dal cammino ottimo, perché ci manca il costo del tratto da (i, j) fino a (m, n) .

Fase 2 - Inversione

Si inverte il grafo e si calcolano i costi $g(i, j)$, cioè da (i, j) a (m, n) , sempre con due righe per volta e spazio ridotto.

Fase 2 - Inversione

Si inverte il grafo e si calcolano i costi $g(i, j)$, cioè da (i, j) a (m, n) , sempre con due righe per volta e spazio ridotto.

Fase 3 - Taglio ottimo

Per ogni riga i , si somma:

$$f(i, j) + g(i, j)$$

Perché non si può continuare senza inversione

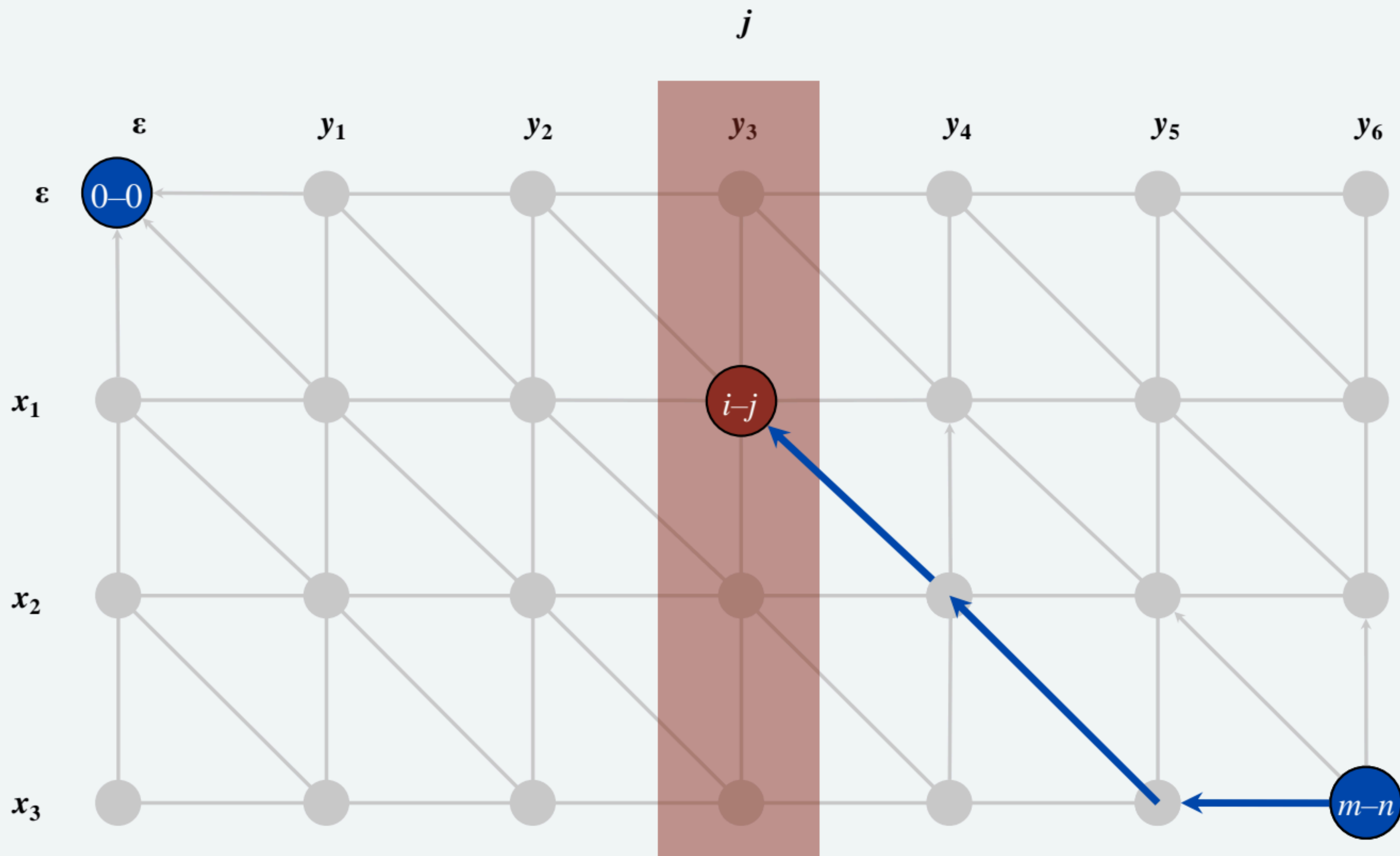
Senza invertire, dovresti calcolare un percorso separato da ogni (i, j) - costi troppo alti. Invertendo, ottieni tutti i $g(i, j)$ in un colpo solo.

↑
PK DIVIDON 2?
↓
PER FARE LA SECONDA
META' METTE CHE $m-n = \text{INIZIO}$
E CHE $j-j = \text{FINE}$
INVERTE GLI ARCHI

Hirschberg's algorithm

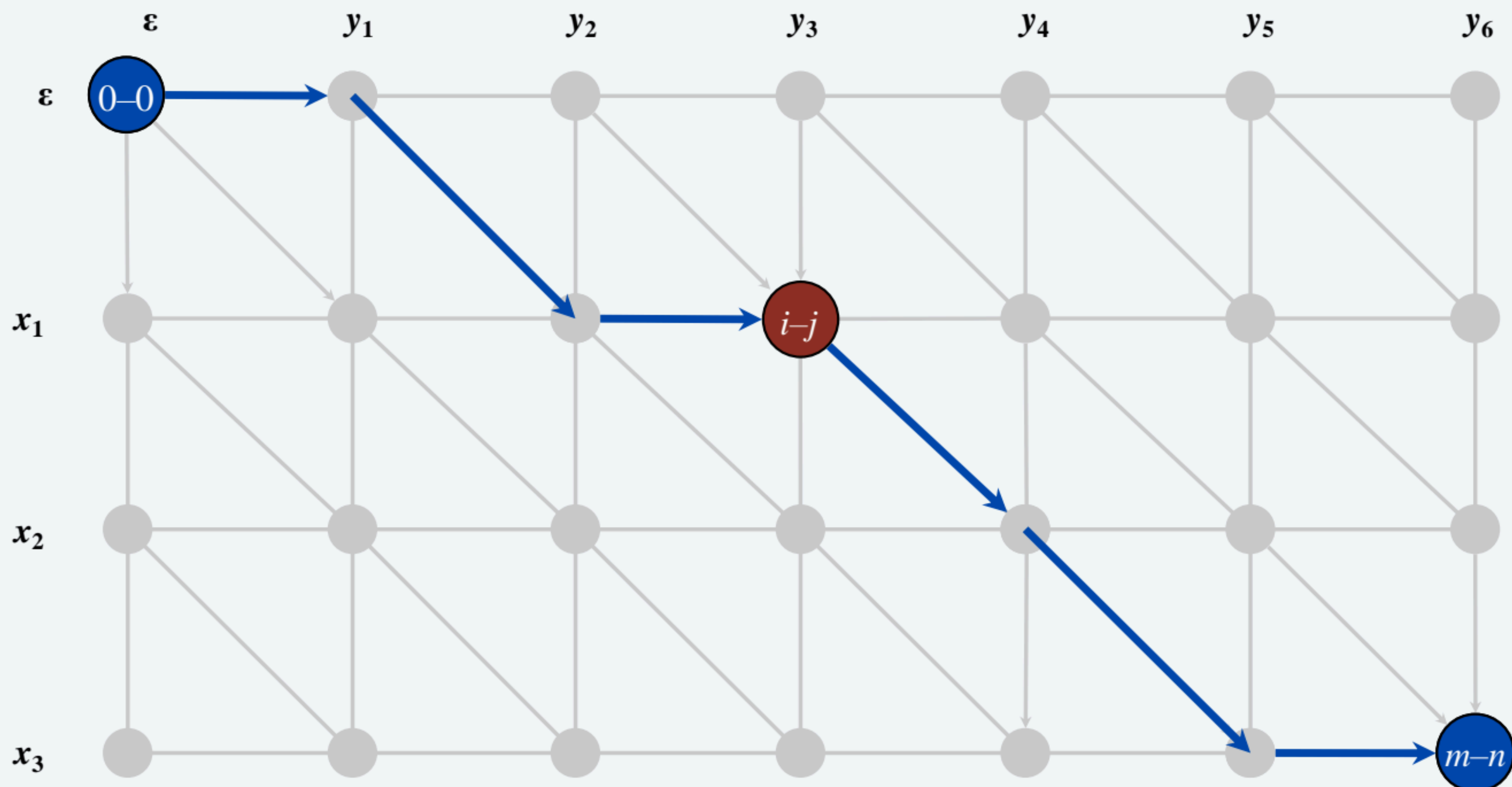
Edit distance graph.

- Let $g(i, j)$ denote length of shortest path from (i, j) to (m, n) .
- Can compute $g(\cdot, j)$ for any j in $O(mn)$ time and $O(m + n)$ space.



Hirschberg's algorithm

Observation 1. The length of a shortest path that uses (i, j) is $f(i, j) + g(i, j)$.



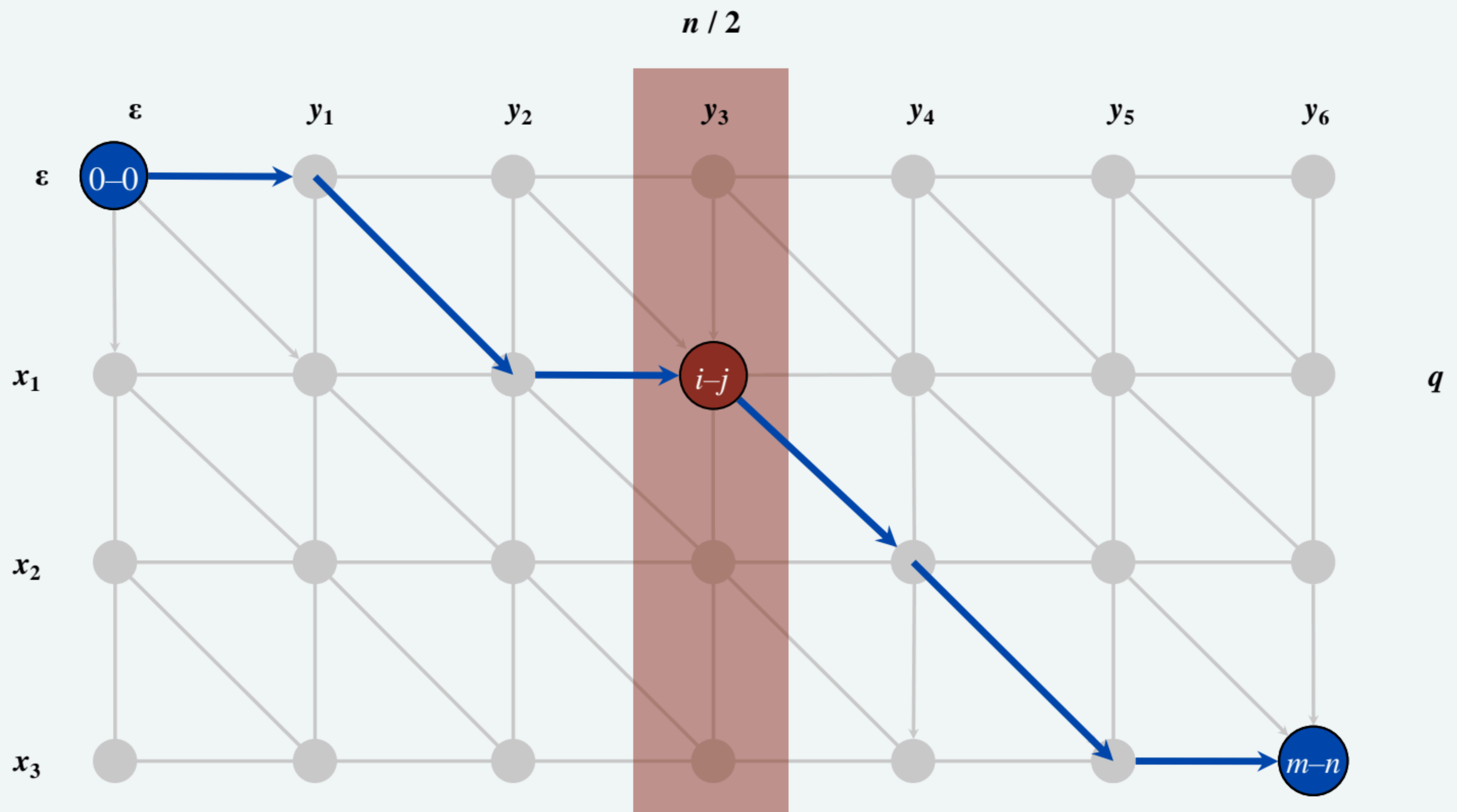
Hirschberg's algorithm

PERCORSO MINIMO

Observation 2. let q be an index that minimizes $f(q, n/2) + g(q, n/2)$.

Then, there exists a shortest path from $(0, 0)$ to (m, n) that uses $(q, n/2)$.

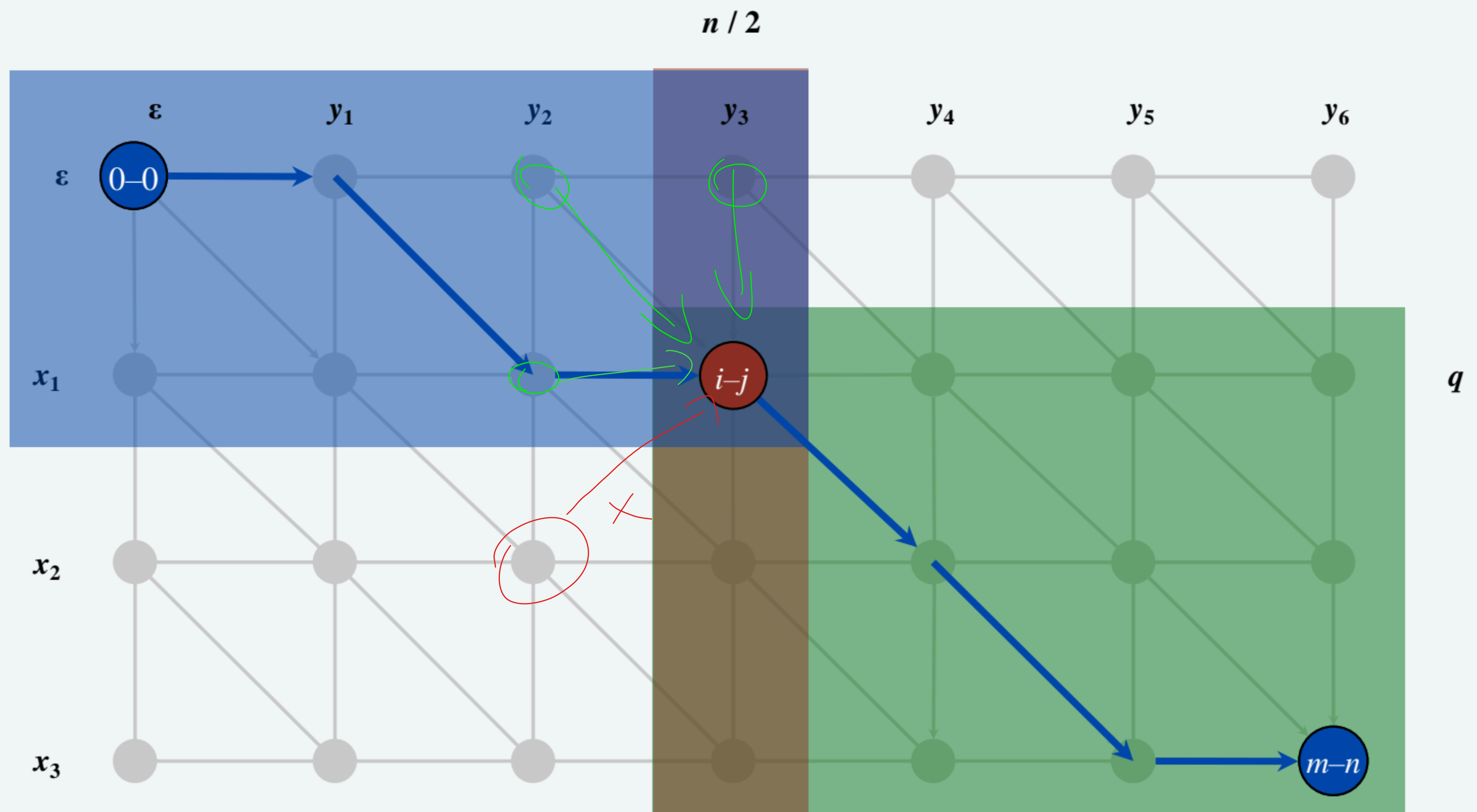
$q = \text{RIGA MINIMA DA PRENDERE}$



Hirschberg's algorithm

Divide. Find index q that minimizes $f(q, n/2) + g(q, n/2)$; save node $i-j$ as part of solution.

Conquer. Recursively compute optimal alignment in each piece.



Hirschberg's algorithm: space analysis

Theorem. Hirschberg's algorithm uses $\Theta(m + n)$ space.

Pf.

- Each recursive call uses $\Theta(m)$ space to compute $f(\cdot, n/2)$ and $g(\cdot, n/2)$.
- Only $\Theta(1)$ space needs to be maintained per recursive call.
- Number of recursive calls $\leq n$. ▪

Hirschberg's algorithm: running time analysis

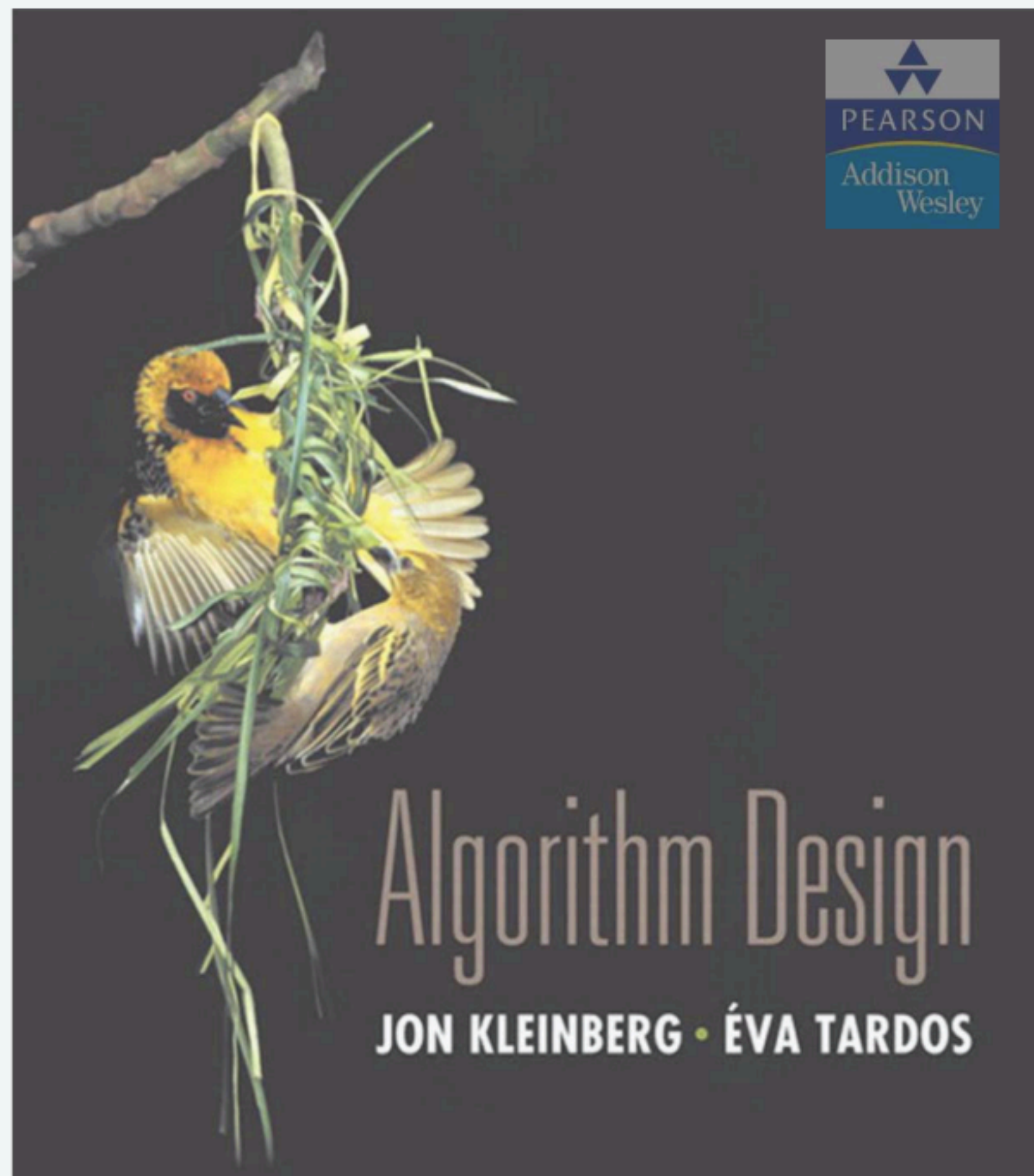
Theorem. Let $T(m, n)$ = max running time of Hirschberg's algorithm on strings of lengths at most m and n . Then, $T(m, n) = O(mn)$.

Anche se divide ricorsivamente, Hirschberg fa al massimo $O(mn)$ lavoro totale. A ogni passo paga cmn , e la somma dei sottoproblemi resta entro $O(mn)$.

Pf. [by strong induction on $m + n$]

- $O(mn)$ time to compute $f(\cdot, n/2)$ and $g(\cdot, n/2)$ and find index q .
- $T(q, n/2) + T(m - q, n/2)$ time for two recursive calls.
- Choose constant c so that:
$$T(m, 2) \leq cm$$
$$T(2, n) \leq cn$$
$$T(m, n) \leq cmn + T(q, n/2) + T(m - q, n/2)$$
- Claim. $T(m, n) \leq 2cmn$.
- Base cases: $m = 2$ and $n = 2$.
- Inductive hypothesis: $T(m', n') \leq 2cm'n'$ for all (m', n') with $m' + n' < m + n$.

$$\begin{aligned} T(m, n) &\leq T(q, n/2) + T(m - q, n/2) + cmn \\ &\stackrel{\text{inductive hypothesis}}{\leq} 2cq n/2 + 2c(m - q) n/2 + cmn \\ &= cq n + cmn - cq n + cmn \\ &= 2cmn \quad \blacksquare \end{aligned}$$



SECTION 6.8

6. DYNAMIC PROGRAMMING II

- ▶ *sequence alignment*
- ▶ *Hirschberg's algorithm*
- ▶ *Bellman–Ford–Moore algorithm*

CAMMIN MININ SINOLA

Shortest paths with negative weights

SORGENTE
SINGOLA
DEST

Shortest-path problem. Given a digraph $G = (V, E)$, with arbitrary edge lengths ℓ_{vw} , find shortest path from source node s to destination node t .

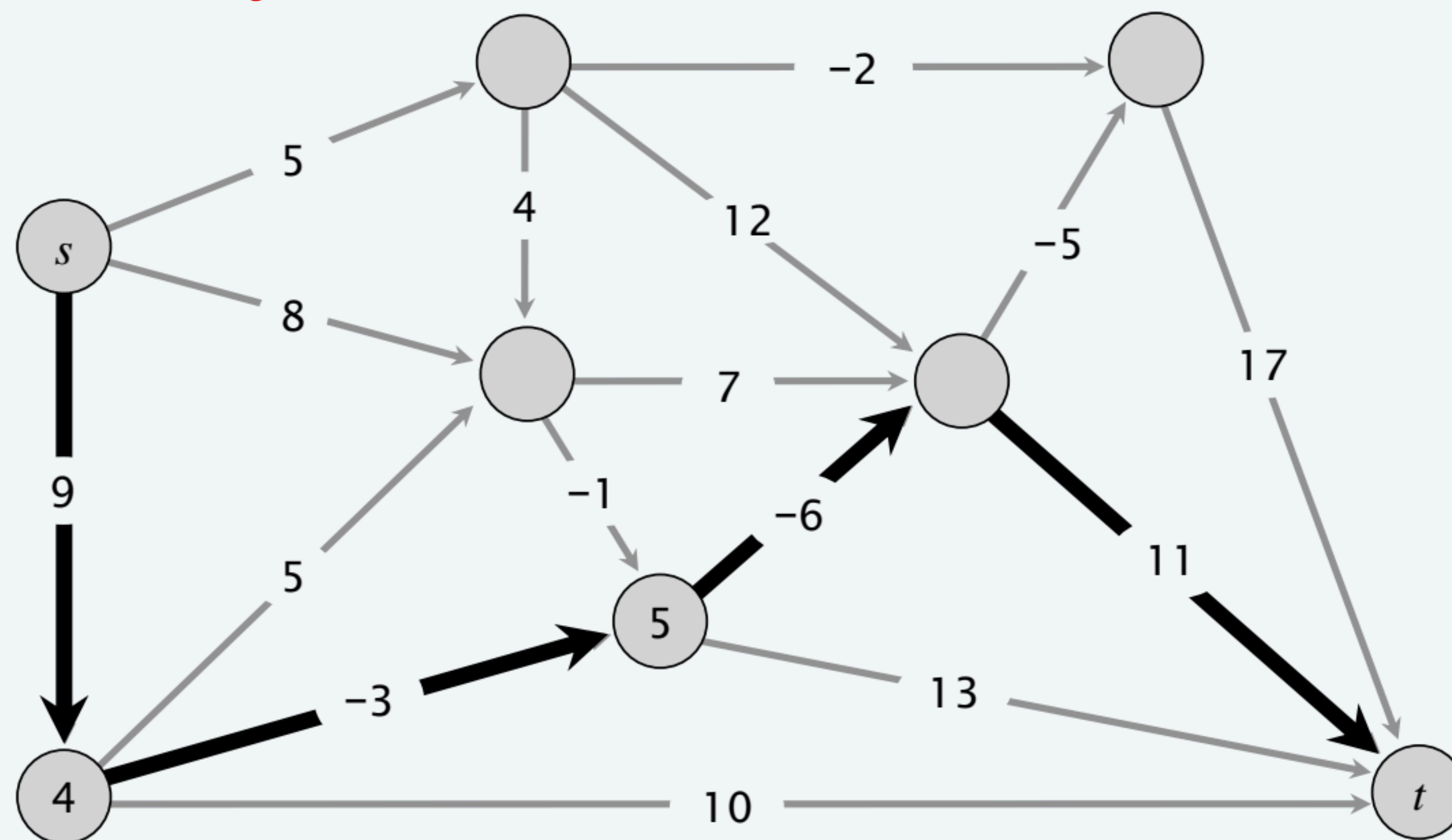
↑
DEFINITO

↑
ANCHE
QUESTO

CON PESI SOLO POSITIVI (≥ 0)

→ USO
DIJKSTRA

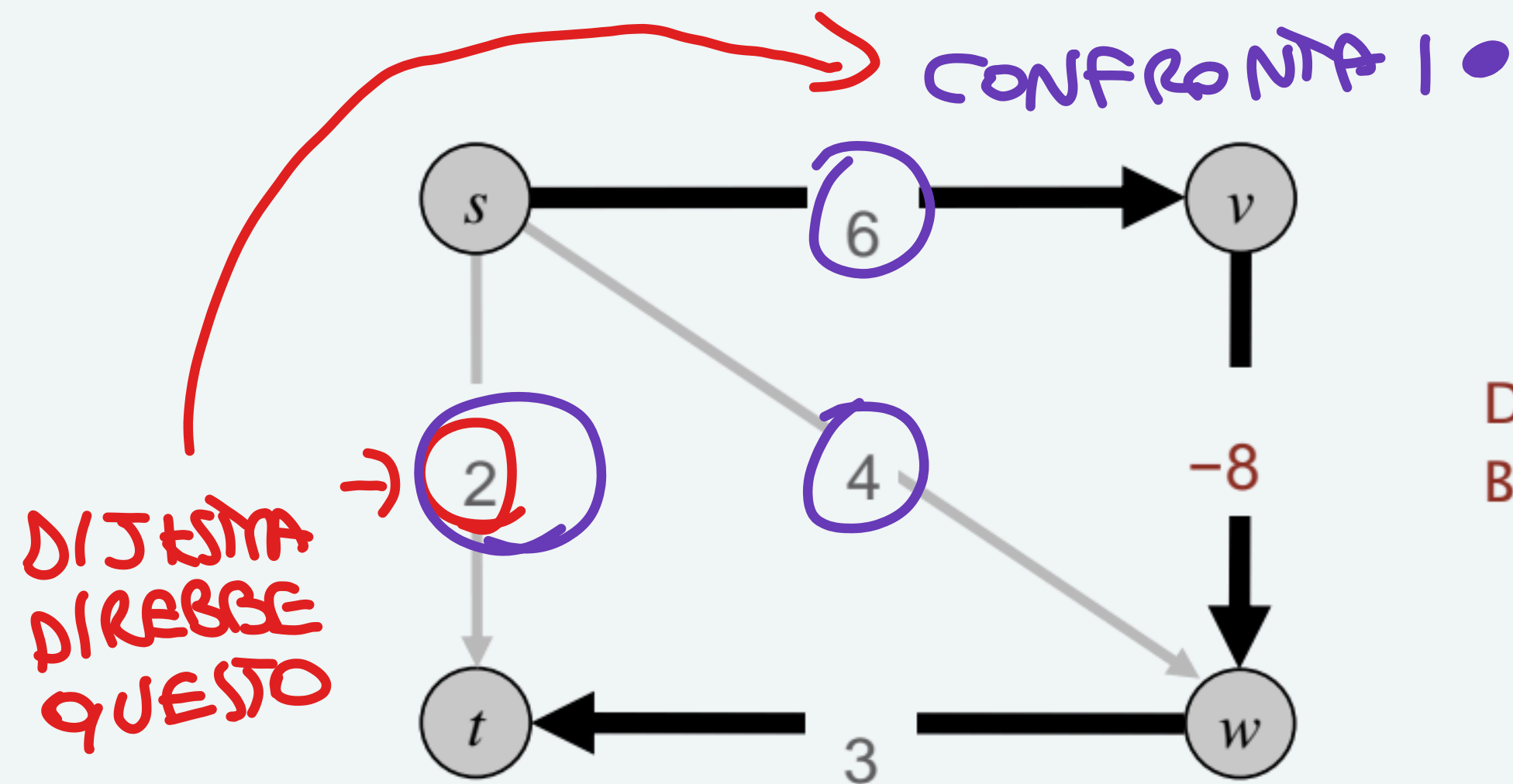
*SENZA → HO BISOGNO DI
ALTRO



length of shortest $s \rightsquigarrow t$ path = $9 - 3 - 6 + 11 = 11$

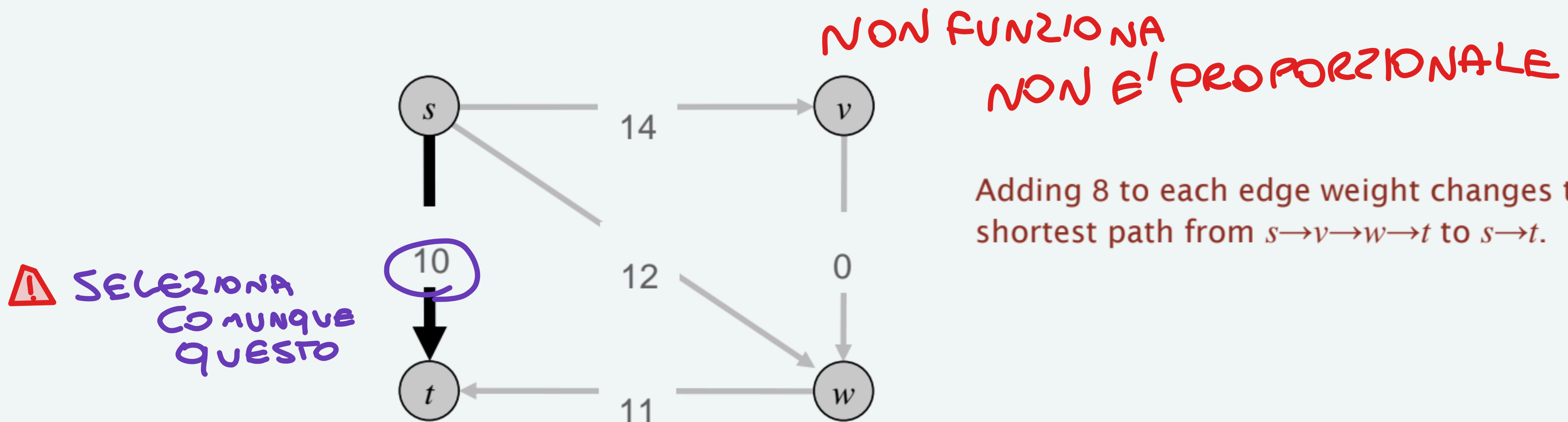
Shortest paths with negative weights: failed attempts

Dijkstra. May not produce shortest paths when edge lengths are negative.



POTREI METTERE TUTTI I PESI CON VALORI POS. → SOMMANDO IL NEGATIVO PIÙ GRANDE

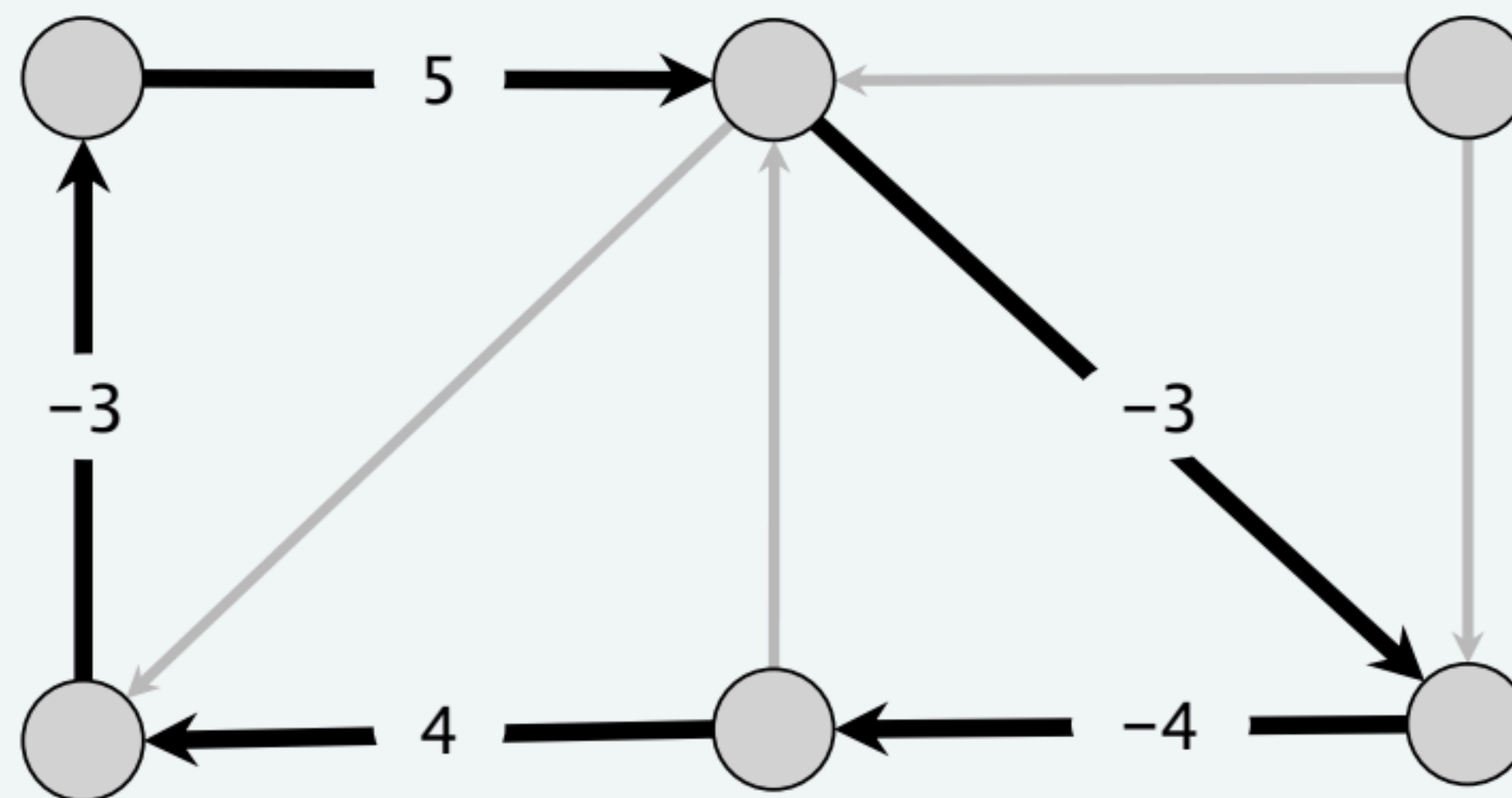
Reweighting. Adding a constant to every edge length does not necessarily make Dijkstra's algorithm produce shortest paths.



Negative cycles

Def. A negative cycle is a directed cycle for which the sum of its edge lengths is negative.

CICLI CON PESI SOMMATI CHE RANNO < 0



! c'è UN PROBLEMA

a negative cycle W :
$$\ell(W) = \sum_{e \in W} \ell_e < 0$$

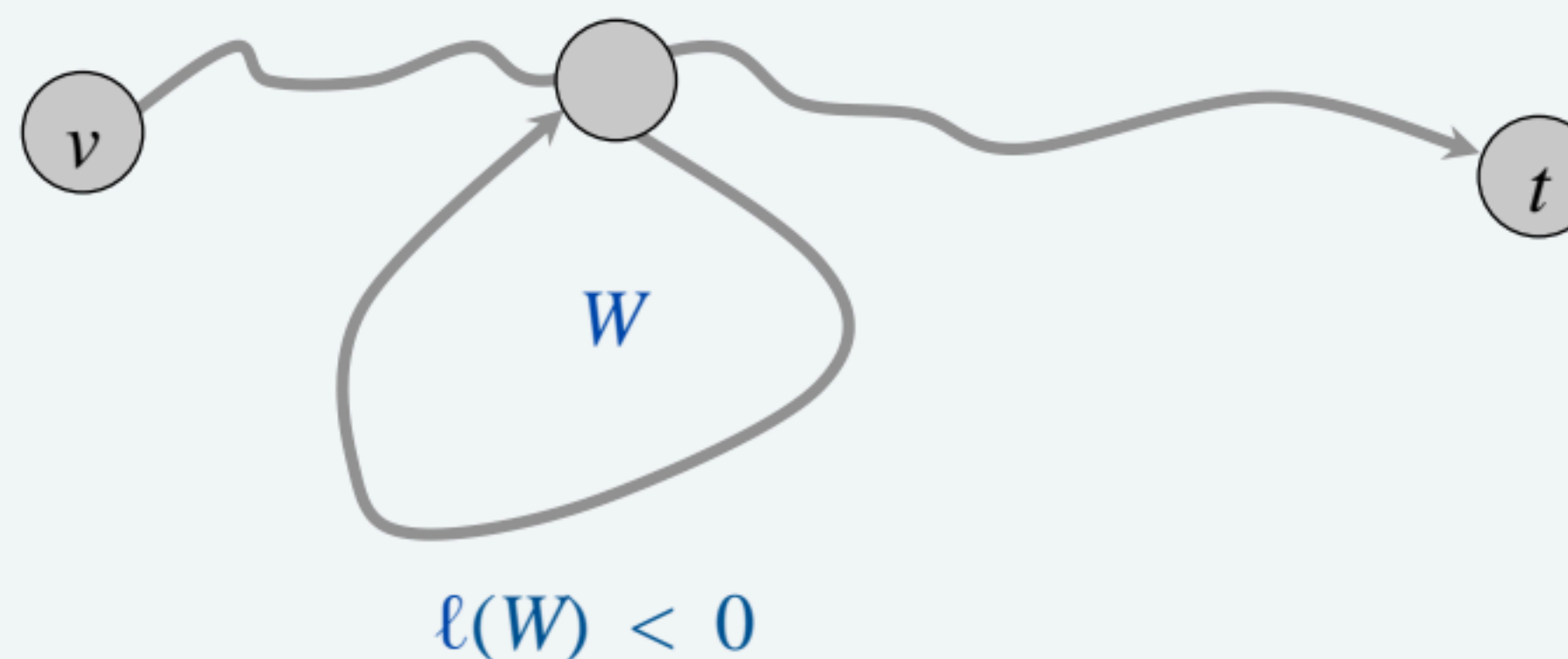
Shortest paths and negative cycles

Lemma 1. If some $v \rightsquigarrow t$ path contains a negative cycle, then there does not exist a shortest $v \rightsquigarrow t$ path.

Proof

Pf. If there exists such a cycle W , then can build a $v \rightsquigarrow t$ path of arbitrarily negative length by detouring around W as many times as desired. ▀

↑
⚠ E' UN PROBLEMA PERCHE' POSSO FARE
IL CICLO QUANTE VOLTE VOGLIO E OGNI
VOLTA RIDURRE IL PESO DEL PERCORSO



Shortest paths and negative cycles

Lemma 2. If G has no negative cycles, then there exists a shortest $v \rightsquigarrow t$ path that is simple (and has $\leq n - 1$ edges).

ARCHI

Pf.

- Among all shortest $v \rightsquigarrow t$ paths, consider one that uses the fewest edges.
- If that path P contains a directed cycle W , can remove the portion of P corresponding to W without increasing its length. ▪



Shortest-paths and negative-cycle problems

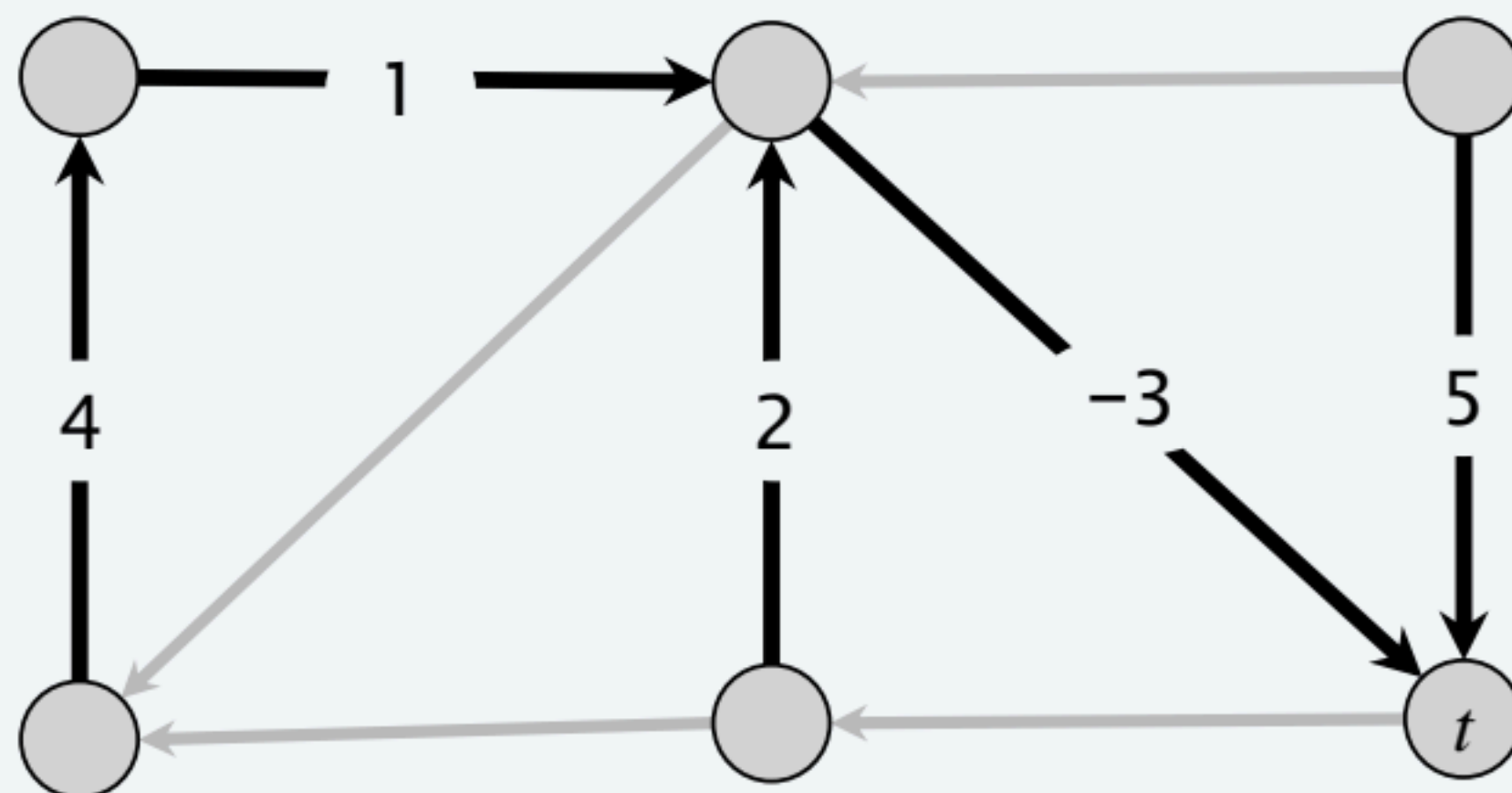
Single-destination shortest-paths problem. Given a digraph $G = (V, E)$ with edge lengths ℓ_{vw} (but no negative cycles) and a distinguished node t , find a shortest $v \rightsquigarrow t$ path for every node v .

CAMMINI MINIMI A SINGOLA DESTINAZIONE (MA NO SORGENTE) ← equivalent to the single-source shortest-paths problem

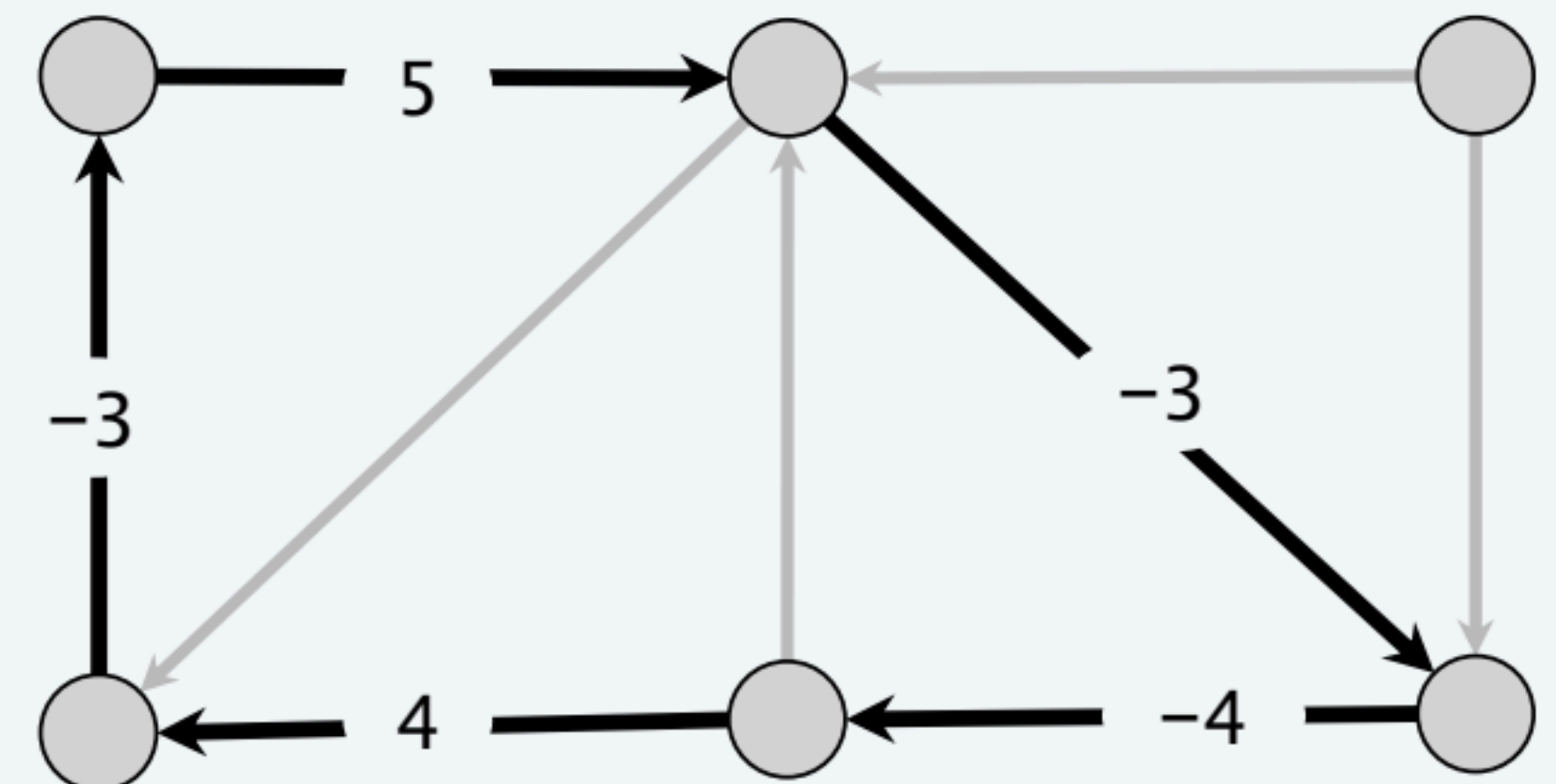
Negative-cycle problem. Given a digraph $G = (V, E)$ with edge lengths ℓ_{vw} , find a negative cycle (if one exists).

allo scorso Sem abbiamo fatto Dijkstra a singola sorgente Ora invece faremo questo algoritmo a singola destinazione è la stessa cosa basta invertire gli archi (edges)

DIJKSTRA = SINGOLA SORG.
QUESTO = SINGOLA DEST.
SONO UGUALI PER BASTA INVERTIRE GLI ARCHI



(reverse) shortest-paths tree



negative cycle

NEGATIVO
SE G HA UN CICLO $< 0 \rightarrow$ ERRORE

SENNO' $\rightarrow$ CAMMINI MINIMI A SINGOLA DEST

Shortest paths with negative weights: dynamic programming

Def. $OPT(i, v)$ = length of shortest $v \rightsquigarrow t$ path that uses $\leq i$ edges.

NUM ARCHI
MASSIMI DA USARE

NODO DI PARTENZA

t E' FISSATO INVECE i E v
NO

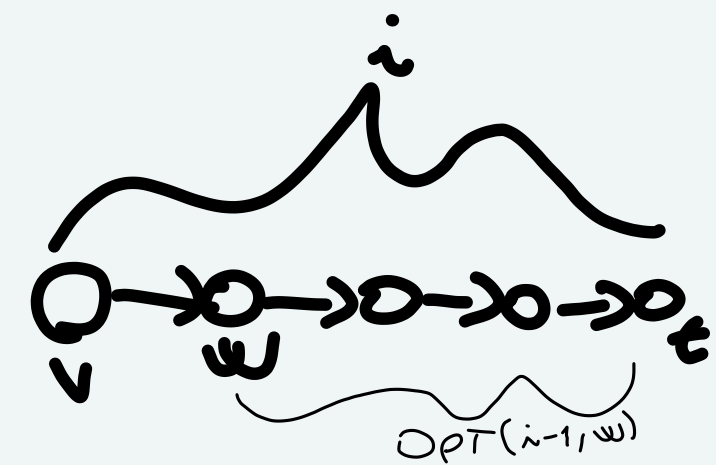
Goal. $OPT(n-1, v)$ for each v .

by Lemma 2, if no negative cycles,
there exists a shortest $v \rightsquigarrow t$ path that is simple

ABBIAMO ISOLITI 2 CASI

Case 1. Shortest $v \rightsquigarrow t$ path uses $\leq i-1$ edges.

- $OPT(i, v) = OPT(i-1, v)$.



$l(v,w) + OPT(i-1, w)$

optimal substructure property
(proof via exchange argument)

Case 2. Shortest $v \rightsquigarrow t$ path uses exactly i edges.

- if (v, w) is first edge in shortest such $v \rightsquigarrow t$ path, incur a cost of l_{vw} .
- Then, select best $w \rightsquigarrow t$ path using $\leq i-1$ edges.

Bellman equation.

$$OPT(i, v) = \begin{cases} 0 & \text{if } i = 0 \text{ and } v = t \\ \infty & \text{if } i = 0 \text{ and } v \neq t \\ \min \left\{ \underset{\text{CASO 1}}{OPT(i-1, v)}, \min_{(v,w) \in E} \left\{ \underset{\text{CASO 2}}{OPT(i-1, w) + l_{vw}} \right\} \right\} & \text{if } i > 0 \end{cases}$$

SITROVA GIÀ LÌ

NON PUO' RAGGIUNGERLO

CASO 1

CASO 2

MIN
PERCHE' S HORT EST

Shortest paths with negative weights: implementation

SHORTEST-PATHS(V, E, ℓ, t)

FOREACH node $v \in V$:

$M[0, v] \leftarrow \infty.$ $\leftarrow$ CASO BASE
← LUNGO O MASSIMO IN V NON POTRO' MAI RAGGIUNGERLO

$M[0, t] \leftarrow 0.$ $\leftarrow$ CASO BASE

FOR $i = 1$ TO $n - 1$ $\leftarrow$ DEFINIRMA $n = \# \text{ nodi}$
INSIEME VERTICI

FOREACH node $v \in V$:

$M[i, v] \leftarrow M[i - 1, v].$

FOREACH edge $(v, w) \in E$:

$M[i, v] \leftarrow \min \{ M[i, v], M[i - 1, w] + \ell_{vw} \}.$
INSIEME ARCHI

Shortest paths with negative weights: implementation

Theorem 1. Given a digraph $G = (V, E)$ with no negative cycles, the DP algorithm computes the length of a shortest $v \rightsquigarrow t$ path for every node v in $\Theta(mn)$ time and $\Theta(n^2)$ space.

Pf.

- Table requires $\Theta(n^2)$ space.
- Each iteration i takes $\Theta(m)$ time since we examine each edge once.

CONFERIRE INFO?
Finding the shortest paths.

- Approach 1: Maintain $successor[i, v]$ that points to next node on a shortest $v \rightsquigarrow t$ path using $\leq i$ edges.
- Approach 2: Compute optimal lengths $M[i, v]$ and consider only edges with $M[i, v] = M[i - 1, w] + \ell_{vw}$. Any directed path in this subgraph is a shortest path.

Durante il calcolo della tabella $M[i, v]$, per ogni nodo v e per ogni passo i , se trovi un miglioramento usando un arco $v \rightarrow w$, ti salvi w come successore di v .

Come funziona:

- All'inizio imposti: $successor[i, v] = null$
- Se scopri che passando per w ottieni un valore minore, salvi:

$successor[i, v] = w$

FA
SCHIARO

Se in qualche iterazione ottieni:
 $M[i, v] = M[i - 1, w] + \ell_{vw}$

Come lo usi?

Alla fine, per ricostruire il cammino da v a t :

- Parti da v
- Cerca w tale che:
 $M[i, v] = M[i - 1, w] + \ell_{vw}$

Quindi: esamini solo gli archi che soddisfano questa uguaglianza, e costruisci il cammino in maniera "retrospettiva" (senza salvarlo durante l'algoritmo).

$$OPT(i, v) = \begin{cases} 0 \\ \infty \\ \min \left\{ OPT(i-1, v), \min_{(v,w) \in E} \{ OPT(i-1, w) + \ell_{vw} \} \right\} \end{cases}$$

USAMO QUESTO ARCO
IN QUESTO IMPLICITAMENTE
SALVANDO TUTTI GLI ARCHI

USIAMO
PRINCIPALMENTE
QUESTO

MIGLIORIE

Shortest paths with negative weights: practical improvements

Space optimization. Maintain two 1D arrays (instead of 2D array).

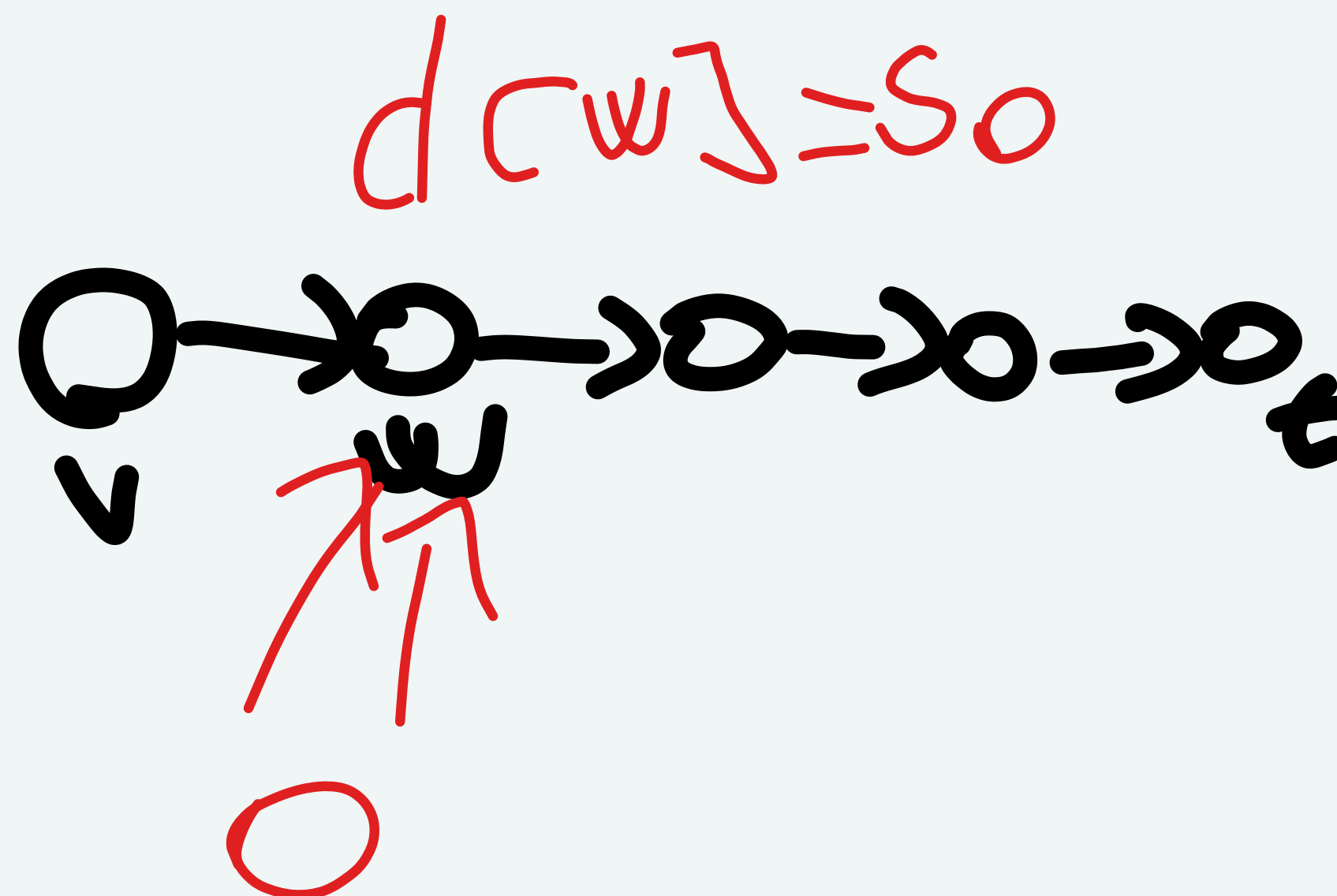
- $d[v]$ = length of a shortest $v \rightsquigarrow t$ path that we have found so far. LUNGH. MIGLIOR CAMMINO FINO A QUEL MOMENTO
- $successor[v]$ = next node on a $v \rightsquigarrow t$ path. STIMA CHE SCENDE $\geq$ RISULTATO

Performance optimization. If $d[w]$ was not updated in iteration $i - 1$, then no reason to consider edges entering w in iteration i .

Per migliorare l'efficienza in termini di memoria, si utilizza l'osservazione che per calcolare $M[i][v]$, è sufficiente conoscere $M[i-1][w]$ per tutti i nodi w adiacenti a v . Pertanto, è possibile mantenere solo due array monodimensionali:

- $d[v]$ contiene le distanze minime aggiornate nell'iterazione corrente.
- $successor[v]$ memorizza, per ogni nodo v , il nodo w successivo nel cammino minimo da v a t .

Durante ogni iterazione, si aggiornano questi array confrontando i valori correnti con quelli dell'iterazione precedente e scegliendo il minimo.



Bellman–Ford–Moore: efficient implementation

DARICORDARE ✓

BELLMAN–FORD–MOORE(V, E, ℓ, t)

FOREACH node $v \in V$:

$d[v] \leftarrow \infty$.

$successor[v] \leftarrow null$.

$d[t] \leftarrow 0$.

FOR $i = 1$ TO $n - 1$

FOREACH node $w \in V$:

IF ($d[w]$ was updated in previous pass)

FOREACH edge $(v, w) \in E$: 

IF ($d[v] > d[w] + \ell_{vw}$)

$d[v] \leftarrow d[w] + \ell_{vw}$.

$successor[v] \leftarrow w$. 

IF (no $d[\cdot]$ value changed in pass i) STOP.

 SENESSUN VALORE d CAMBIA STOP

pass i
 $O(m)$ time

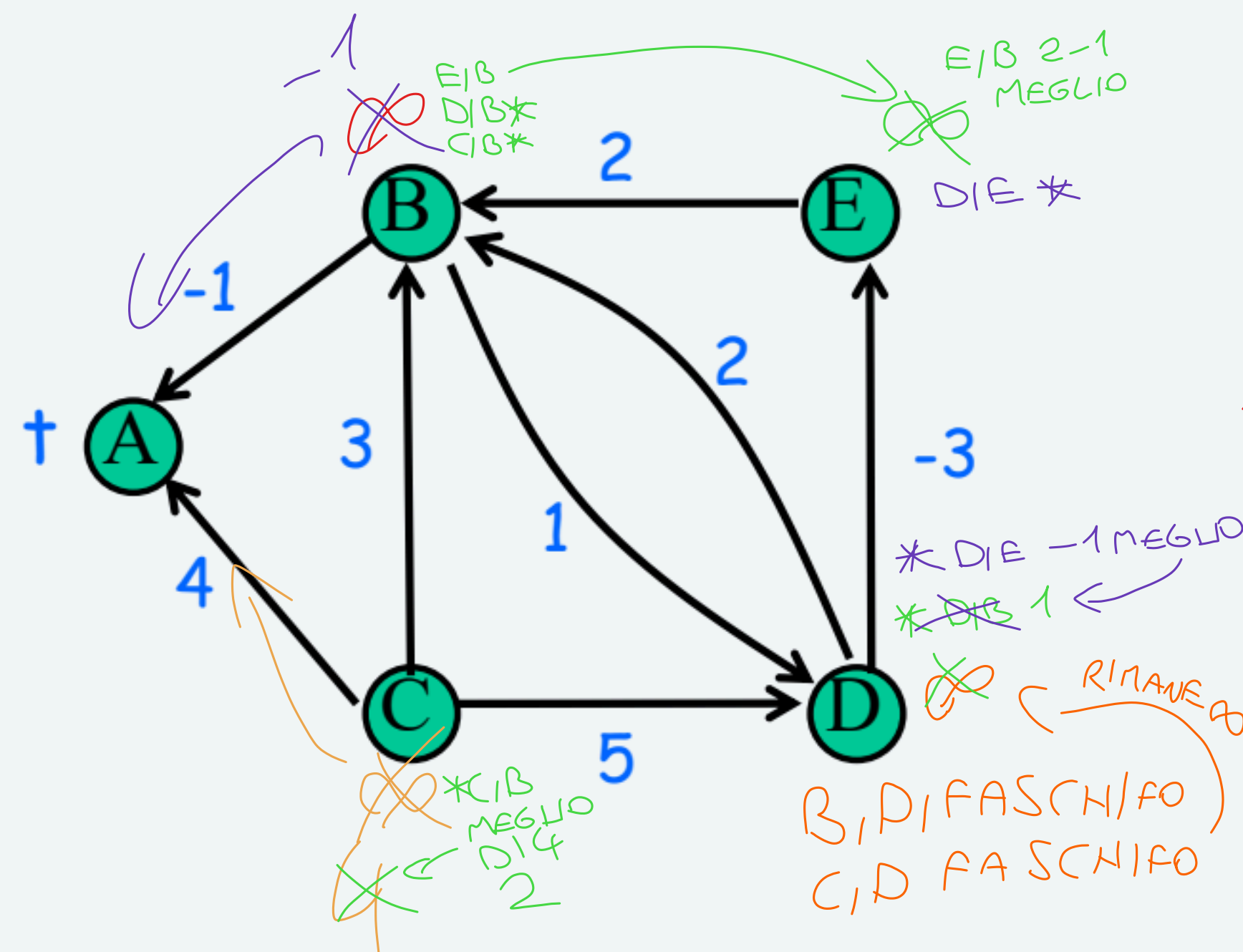
"qualche valore dentro l'array d è cambiato?"
In pratica:
• $d[v]$ è la distanza minima trovata da ogni nodo v a t .
• Durante ogni passaggio (i -esima iterazione) del ciclo principale, controlla se almeno uno di questi valori è cambiato.

Se nessuno è cambiato, allora vuol dire che non troverai più cammini migliori nei prossimi passaggi. Quindi puoi interrompere l'algoritmo in anticipo.

Example: try to run Bellman-Ford algorithm on the following graph.

Order of the nodes: t, D, C, B, E

$$\# \text{ PASSATE} = n - 1 = 4$$



PONIAMO UN ORDINE DEI NODI
 1° PASSATA
 A / D / C / B / E
 $w=A$ $w=D$ $w=C$ $w=B$ $w=E$
 VEDO TUTTI GLI ARCHI ENTRANTI DI D
 NO ARCHI ENTR

2° PASSATA VEDENDO i d[u]?

A / D / C / B / E
 // // // // //
 GIÀ VISTA NON CAMBIA NULLA NULLA NULLA NULLA
 3° PASSATA DELLA PASSATA 2 NON CAMBIA NULLA NULLA
 QUINDI NON LA FACILIO
 4° NEMMENO

1. le distanze sono tutte ∞ TRanne $t=0$

Bellman–Ford–Moore: analysis

Lemma 3. For each node v : $d[v]$ is the length of some $v \rightsquigarrow t$ path. SONO STIME CON CAMMINI REALI

Lemma 4. For each node v : $d[v]$ is monotone non-increasing. le STIME le cambiano solo se migliorano

NON ABBIAMO MAI SOVRASTIME AL PASSO i -esimo

DOPO UNA PASSATA i

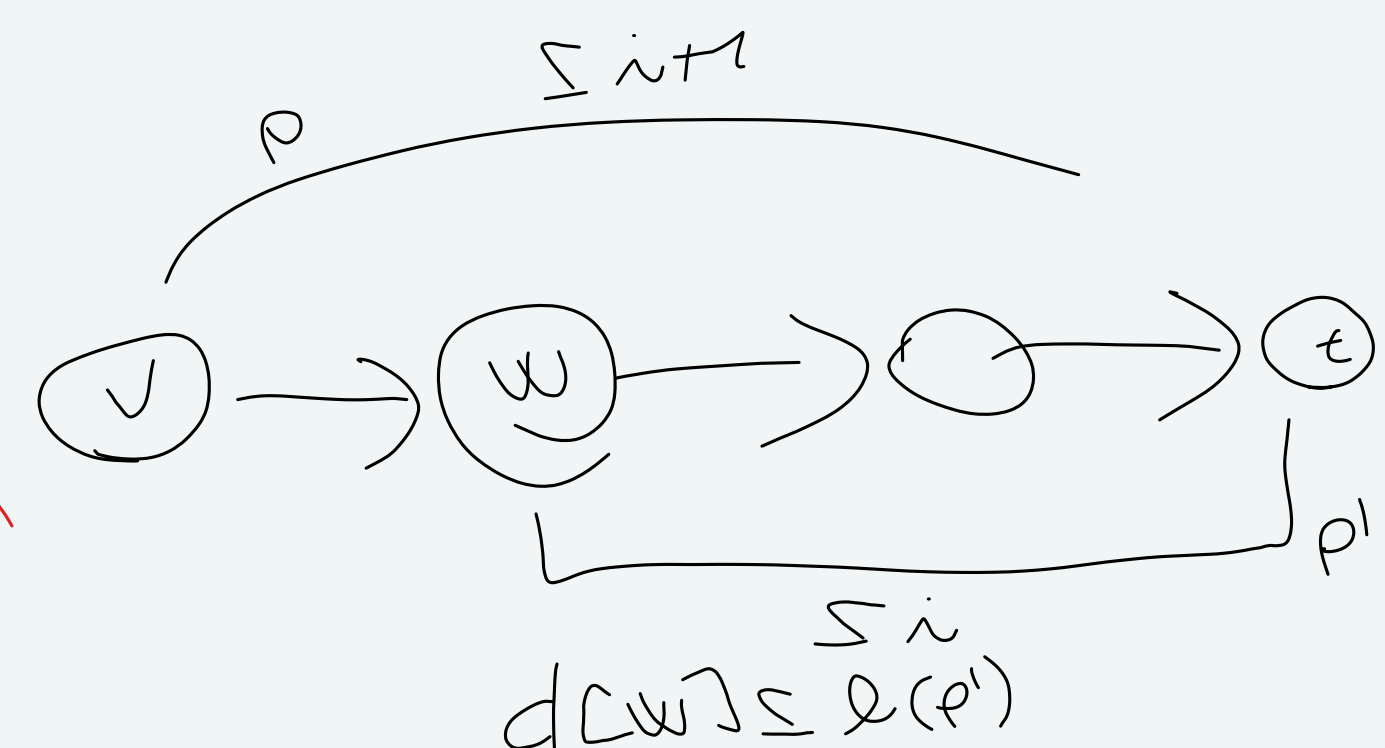
Lemma 5. After pass i , $d[v] \leq$ length of a shortest $v \rightsquigarrow t$ path using $\leq i$ edges. CAMMINO (HEC) ANO AL PIU' i ARCHI

Pf. [by induction on i]

- Base case: $i = 0$.
- Assume true after pass i .
- Let P be any $v \rightsquigarrow t$ path with $\leq i + 1$ edges.
- Let (v, w) be first edge in P and let P' be subpath from w to t .
- By inductive hypothesis, at the end of pass i , $d[w] \leq \ell(P')$ because P' is a $w \rightsquigarrow t$ path with $\leq i$ edges.
- After considering edge (v, w) in pass $i + 1$:

$$\begin{aligned}
 d[v] &\leq \ell_{vw} + d[w] \\
 &\leq \ell_{vw} + \ell(P') \\
 &= \ell(P) \quad \blacksquare
 \end{aligned}$$

and by Lemma 4, $d[v]$ does not increase



Lemma 5 (in parole povere)
Dopo la passata numero i dell'algoritmo:
Per ogni nodo v , il valore salvato in $d[v]$ è al massimo la lunghezza del cammino più corto da v a t che usa al massimo i archi.

Ma perché è importante questo?
Perché Bellman-Ford aggiorna i valori passo dopo passo, tenendo in considerazione cammini sempre più lunghi (prima cammini con 1 arco, poi con 2 archi, ..., fino a i).
Questo lemma ti dice che, non importa che sia avanti, non stai mai sovrestimando la vera distanza. Il valore in $d[v]$ è sempre il migliorabile, ma mai peggiore di quanto potrebbe essere.

La dimostrazione (Proof) in parole semplici
La dimostrazione è per induzione su i (cioè il numero di passaggi fatti).

1. Caso base: $i = 0$
• All'inizio (passo 0), sai che $d[t] = 0$ (perché vuoi arrivare a te stesso) e tutti gli altri $d[v] = \infty$.
• È vero che $d[v]$ rappresenta la distanza più breve usando 0 archi? Sì, perché puoi arrivare solo a te stesso senza usare archi, e a tutti gli altri non puoi arrivare (quindi infinito).
... il lemma è vero per $i = 0$.

2. Passo induttivo: supponiamo che il lemma sia vero per i , vogliamo mostrarlo per $i + 1$.
Cioè:
Supponiamo che dopo il passo i , $d[v]$ è corretto per tutti i cammini con al massimo i archi.
• Aggiorna $d[v]$ a $d[v] + \ell(v, w)$.
Ma per ipotesi induttiva, dopo il passo i sappiamo che $d[w]$ è già minimo per tutti i cammini da w a t con al massimo i archi.
Quindi:
 $d[v] \leq d[v] + \ell(v, w) \leq$ lunghezza del cammino $v \rightarrow w \rightarrow t$.
Questo vuol dire che dopo il passo $i + 1$ ha calcolato un valore per $d[v]$ che è corretto (cioè non sovrastima) per cammini che usano al massimo $i + 1$ archi.

Bellman–Ford–Moore: analysis

Theorem 2. Assuming no negative cycles, Bellman–Ford–Moore computes the lengths of the shortest $v \rightsquigarrow t$ paths in $O(mn)$ time and $\Theta(n)$ extra space.

Pf. Lemma 2 + Lemma 5. ▀

shortest path exists and
has at most $n-1$ edges

after i passes,
 $d[v] \leq$ length of shortest path
that uses $\leq i$ edges

Remark. Bellman–Ford–Moore is typically faster in practice.

- Edge (v, w) considered in pass $i + 1$ only if $d[w]$ updated in pass i .
- If shortest path has k edges, then algorithm finds it after $\leq k$ passes.

now don't have performance $n-1$ max k
solo allebrute

what about the shortest paths?

CONELI PRENDI?



Bellman-Ford-Moore: analysis

RIPERCORO TUTTI I SUCC.DIV

Claim. Throughout Bellman-Ford-Moore, following the $\text{successor}[v]$ pointers gives a directed path from v to t of length $d[v]$.

Counterexample. Claim is false!

- Length of successor $v \rightsquigarrow t$ path may be strictly shorter than $d[v]$.

ESEMPLO

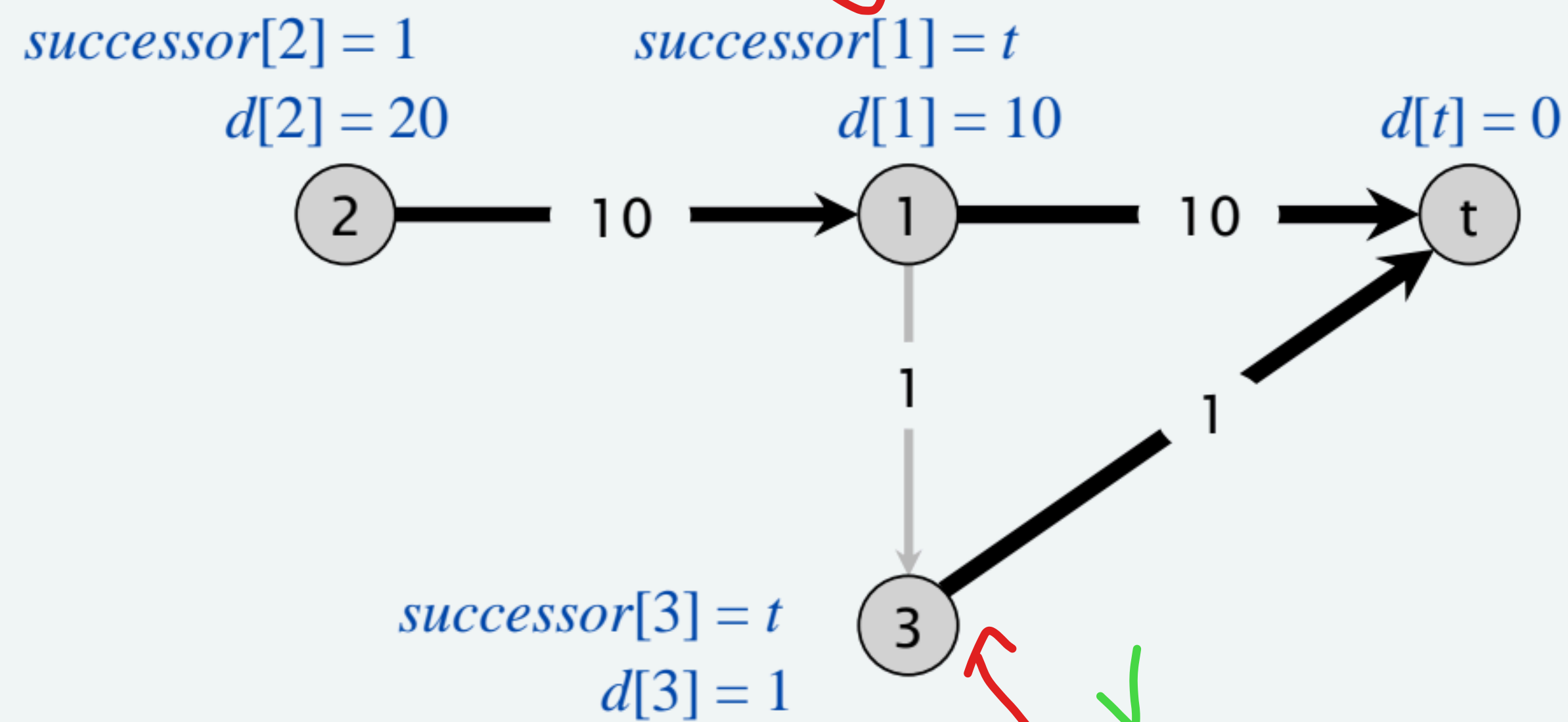
i successori non vengono sempre aggiornati quindi in questo esempio $\text{successor}[1]$ ci riporta a t mentre invece dovrebbe portarci al nodo 3

consider nodes in order: $t, 1, 2, 3$

Domanda cruciale: perché non viene aggiornato $\text{successor}[1]$ a 3?

Perché l'algoritmo guarda gli archi entranti per ogni nodo, e aggiorna solo se trova un costo migliore.

- Ma $d[1] = 10$
- $3 \rightarrow 1$ non esiste (nell'esempio)
- Quindi non c'è nessuna occasione di aggiornare $d[1]$
- E se non aggiorni $d[1]$, allora non aggiorni $\text{successor}[1]$

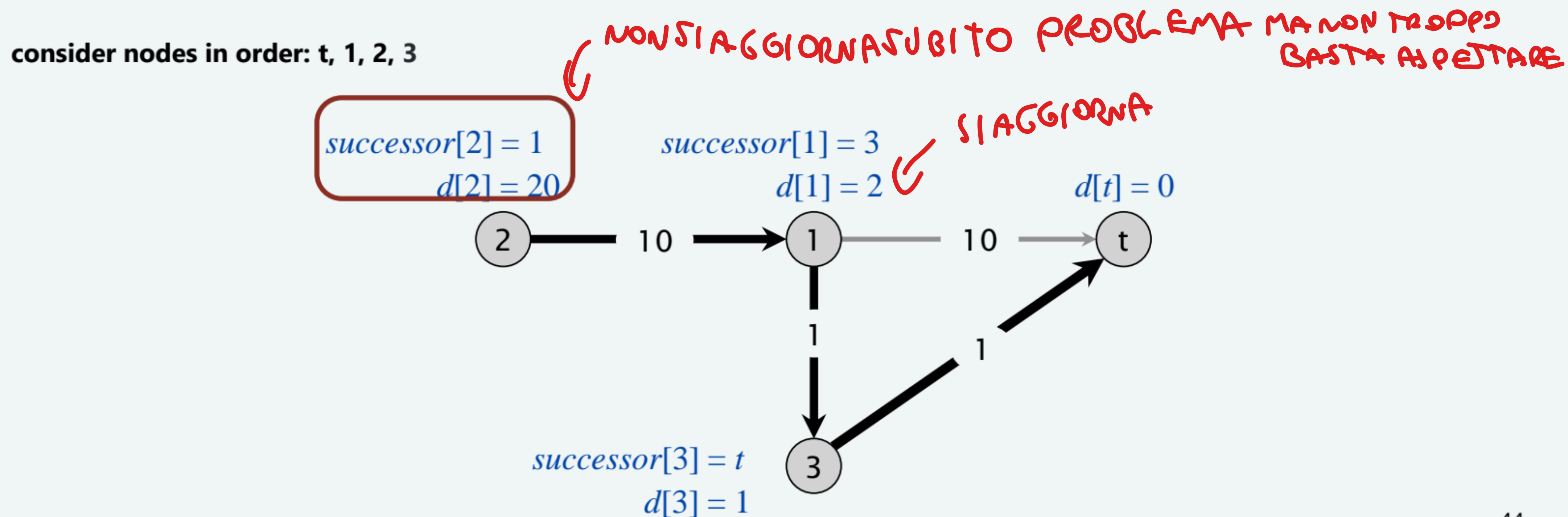


Bellman–Ford–Moore: analysis

Claim. ~~Throughout Bellman–Ford–Moore, following the $\text{successor}[v]$ pointers gives a directed path from v to t of length $d[v]$.~~

Counterexample. Claim is false!

- Length of successor $v \rightsquigarrow t$ path may be strictly shorter than $d[v]$.



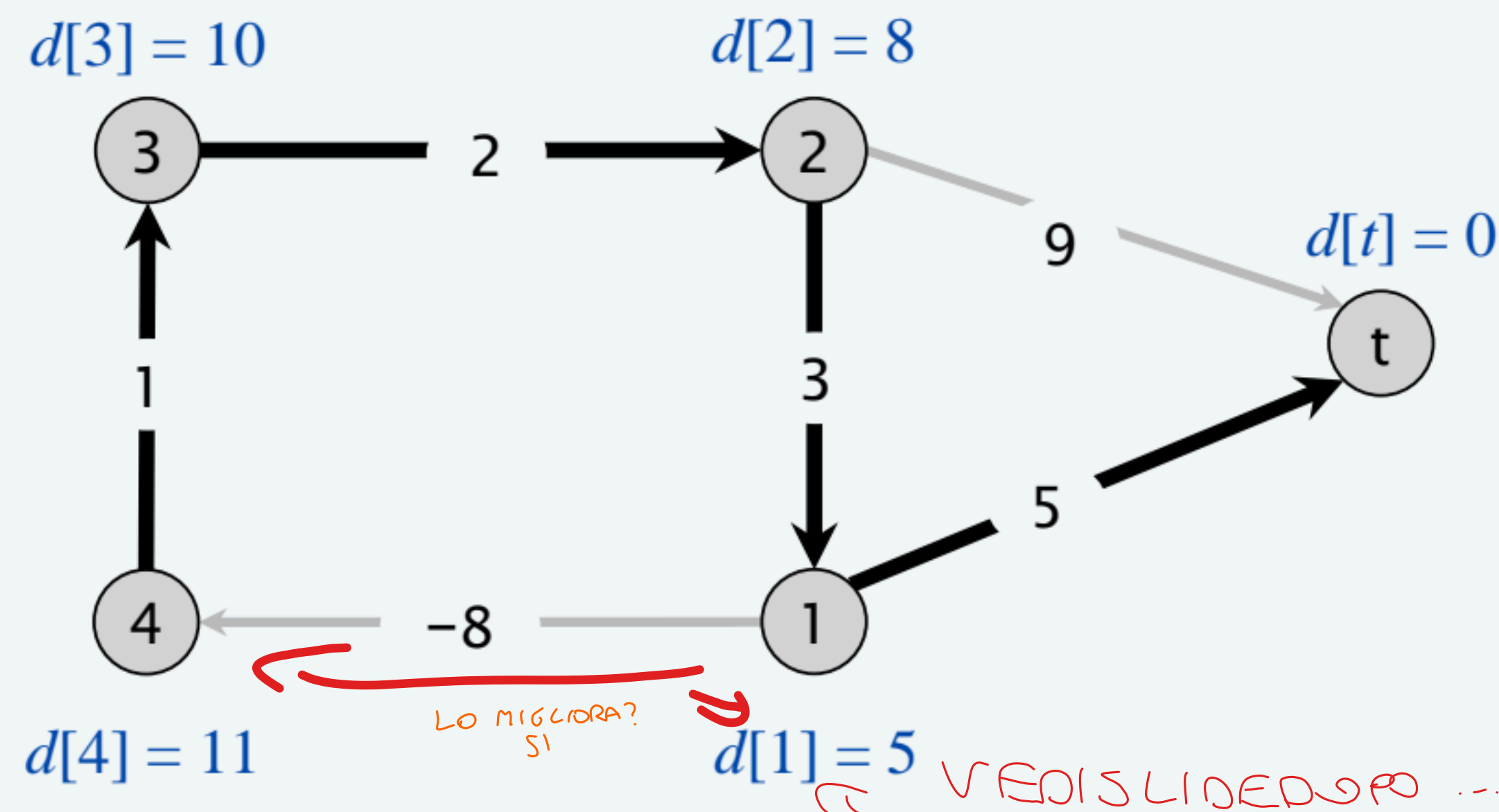
Bellman–Ford–Moore: analysis

Claim. ~~Throughout Bellman–Ford–Moore, following the $\text{successor}[v]$ pointers gives a directed path from v to t of length $d[v]$.~~

Counterexample. Claim is false!

- Length of successor $v \rightsquigarrow t$ path may be strictly shorter than $d[v]$.
- If negative cycle, successor graph may have directed cycles.

consider nodes in order: $t, 1, 2, 3, 4$



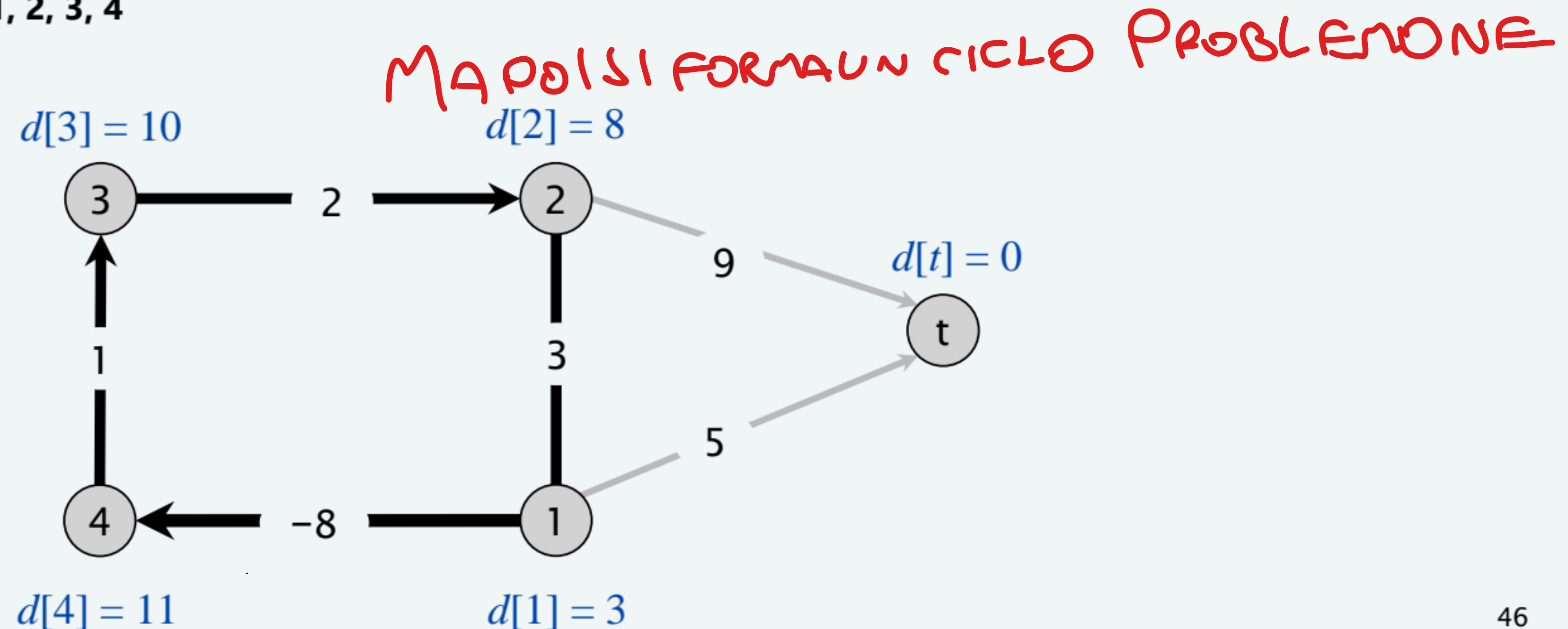
Bellman–Ford–Moore: analysis

Claim. ~~Throughout Bellman–Ford–Moore, following the $\text{successor}[v]$ pointers gives a directed path from v to t of length $d[v]$.~~

Counterexample. Claim is false!

- Length of successor $v \rightsquigarrow t$ path may be strictly shorter than $d[v]$.
- If negative cycle, successor graph may have directed cycles.

consider nodes in order: $t, 1, 2, 3, 4$



Bellman–Ford–Moore: finding the shortest paths

Lemma 6. Any directed cycle W in the successor graph is a negative cycle.

Pf.

OGNICOLO E' NEG.

si dimostra matematicamente che se l'algoritmo produce un ciclo nei successori allora quel ciclo è negativo

- If $\text{successor}[v] = w$, we must have $d[v] \geq d[w] + \ell_{vw}$.
(LHS and RHS are equal when $\text{successor}[v]$ is set; $d[w]$ can only decrease; $d[v]$ decreases only when $\text{successor}[v]$ is reset)
- Let $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$ be the sequence of nodes in a directed cycle W .
- Assume that (v_k, v_1) is the last edge in W added to the successor graph.

Just prior to that: $d[v_1] \geq d[v_2] + \ell(v_1, v_2)$

$$d[v_2] \geq d[v_3] + \ell(v_2, v_3)$$

$$\vdots$$

$$d[v_{k-1}] \geq d[v_k] + \ell(v_{k-1}, v_k)$$

$$d[v_k] > d[v_1] + \ell(v_k, v_1)$$

holds with strict inequality since we are updating $d[v_k]$

spiegazione
chat gpt
(informale)

1. Le stime $d[v]$ nel Bellman-Ford possono solo diminuire nel tempo (mai aumentare).
2. Se trovi un ciclo nei successori, significa che:
• Ogni $d[v]$ in quel ciclo è stato aggiornato usando un altro $d[w]$ "più conveniente".
• Ma in un ciclo, stai tornando al nodo di partenza... e se in ogni passo la stima d è migliorata, alla fine torni a un valore più piccolo di quello con cui sei partito.
3. Questo è una contraddizione a meno che... la somma dei pesi nel ciclo sia negativa!

- Adding inequalities yields $\ell(v_1, v_2) + \ell(v_2, v_3) + \dots + \ell(v_{k-1}, v_k) + \ell(v_k, v_1) < 0$. ▪

W is a negative cycle

Bellman–Ford–Moore: finding the shortest paths

dobbiamo per forza assumere che non ci siano cicli negativi per il lemma 6

Theorem 3. Assuming no negative cycles, Bellman–Ford–Moore finds shortest $v \rightsquigarrow t$ paths for every node v in $O(mn)$ time and $\Theta(n)$ extra space.

Pf. PRENDIAMO I SUCCESSORI DOPO AVERLI AGGIORNATI TUTTI

Nella slide 43 si diceva:
"seguire i successor potrebbe dare un cammino sbagliato"
Ma in realtà quello era un controesempio durante l'esecuzione, quando i valori erano ancora in aggiornamento.
Ora invece, siamo a fine algoritmo, e la slide 47 ci ha già assicurato che:

- The successor graph cannot have a directed cycle. [Lemma 6]
- Thus, following the successor pointers from v yields a directed path to t .
- Let $v = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k = t$ be the nodes along this path P .
- Upon termination, if $\text{successor}[v] = w$, we must have $d[v] = d[w] + \ell_{vw}$.
(LHS and RHS are equal when $\text{successor}[v]$ is set; $d[\cdot]$ did not change)

Thus,

$$\begin{array}{rcl} d[v_1] & = & d[v_2] + \ell(v_1, v_2) \\ d[v_2] & = & d[v_3] + \ell(v_2, v_3) \\ \vdots & & \vdots \\ d[v_{k-1}] & = & d[v_k] + \ell(v_{k-1}, v_k) \end{array}$$

SI CANCELLANO

since algorithm terminated

RIMANGONO $d[v_1], d[v_k], \ell(v_{k-1}, v_k)$

- Adding equations yields $d[v] = d[t] + \ell(v_1, v_2) + \ell(v_2, v_3) + \dots + \ell(v_{k-1}, v_k)$. ▪

Riassunto semplice	
Slide	Cosa dimostra
43	Durante l'algoritmo, i $\text{successor}[v]$ possono essere inconsistenti
47	Se c'è un ciclo nei successor , allora è un ciclo negativo
48	Se non ci sono cicli negativi, allora i $\text{successor}[v]$ ti danno davvero il cammino più corto da ogni v a t

min length of any $v \rightsquigarrow t$ path (Theorem 2)

0

RIMANGONO I SUCCESSORI

length of path P

$\phi = d[v]$

IN QUESTO CASO SONO I PEJ! MA ALLA FINE POSSONO ANCHE ESSERE VISTI COME I PERCORSI

what if the graph has a negative cycle (the can reach t)?

BELLMAN–FORD–MOORE(V, E, ℓ, t)

FOREACH node $v \in V$:

$d[v] \leftarrow \infty$.

$successor[v] \leftarrow null$.

$d[t] \leftarrow 0$.

FOR $i = 1$ **TO** $n - 1$

FOREACH node $w \in V$:

IF ($d[w]$ was updated in previous pass)

FOREACH edge $(v, w) \in E$:

IF ($d[v] > d[w] + \ell_{vw}$)

$d[v] \leftarrow d[w] + \ell_{vw}$.

$successor[v] \leftarrow w$.

IF (no $d[\cdot]$ value changed in pass i) **STOP**.

VECCIO

Bellman–Ford–Moore: checking for negative cycle

↑ PER
SPOTTARE
CICLI
NEGATIVI

BELLMAN–FORD–MOORE(V, E, ℓ, t)

FOREACH node $v \in V$:

$d[v] \leftarrow \infty$.

$successor[v] \leftarrow null$.

$d[t] \leftarrow 0$.

FOR $i = 1$ TO $n - 1$

FOREACH node $w \in V$:

IF ($d[w]$ was updated in previous pass)

FOREACH edge $(v, w) \in E$:

IF ($d[v] > d[w] + \ell_{vw}$)

$d[v] \leftarrow d[w] + \ell_{vw}$.

$successor[v] \leftarrow w$.

IF (no $d[\cdot]$ value changed in pass i) STOP.

FOREACH edge $(v, w) \in E$

IF ($d[v] > d[w] + \ell_{vw}$) THEN return “there is a negative cycle”

pass n
 $O(m)$ time

DT CHIAMA SE C'E' UN CICLO NEGATIVO

Perché funziona?

Perché se dopo $n - 1$ iterazioni riesci ancora a migliorare $d[v]$ per qualche nodo v , significa che:

- Hai trovato un percorso ancora più corto,
- Ma il teorema dice che in un grafo senza cicli negativi, il cammino più corto ha al massimo $n - 1$ archi.

Quindi, se dopo $n - 1$ giri puoi ancora migliorare qualcosa, è per forza colpa di un ciclo negativo che può essere percorso più volte per ridurre all'infinito il costo.

DIMOSTRIAMO CHE SE CI SONO CICLI NEGATIVI SE NE ACCORGE

Lemma 6. If there is a negative cycle (that can reach t) the (modified) algorithm report it.

Pf.

- If there is no negative cycle the pass $\#n$ do nothing
- Let $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$ a directed **negative** cycle W .
- assume by contradiction that the algorithm does not return it
- Then: condition of the last **IF** is always false.

▪ Hence:

$$\begin{array}{rcl} d[v_1] & \leq & \cancel{d[v_2]} + \ell(v_1, v_2) \\ \cancel{d[v_2]} & \leq & \cancel{d[v_3]} + \ell(v_2, v_3) \\ \vdots & & \vdots \\ d[v_{k-1}] & \leq & d[v_k] + \ell(v_{k-1}, v_k) \\ d[v_k] & \leq & d[v_1] + \ell(v_k, v_1) \end{array}$$

COA RI MANE?

Struttura della dimostrazione (per assurdo)

1. Ipotesi: c'è un ciclo negativo W :

$$v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$$
$$\ell(W) = \ell(v_1, v_2) + \ell(v_2, v_3) + \dots + \ell(v_k, v_1) < 0$$

1. Supponiamo per assurdo che l'algoritmo non lo rilevi.

– Cioè: l'ultimo controllo della slide 50 non ha mai trovato:

$$d[v] > d[w] + \ell(v, w)$$

1. Quindi, per ogni arco del ciclo W , è vero che:

$$d[v_i] \leq d[v_{i+1}] + \ell(v_i, v_{i+1})$$

Somma tutti i lati sinistri e destri:

- Lato sinistro: $d[v_1] + d[v_2] + \dots + d[v_k]$
- Lato destro: anche $d[v_1] + d[v_2] + \dots + d[v_k]$, più la somma di tutti gli $\ell(v_i, v_{i+1})$

Quando li sottrai da entrambi i lati, rimane:

$$0 \leq \ell(W)$$

Ma c'è un problema

Abbiamo assunto che il ciclo W è negativo:

$$\ell(W) < 0$$

Quindi:

- da una parte stiamo dicendo $0 \leq \ell(W)$
- dall'altra $\ell(W) < 0$

Contraddizione!

- Adding inequalities yields $\ell(v_1, v_2) + \ell(v_2, v_3) + \dots + \ell(v_{k-1}, v_k) + \ell(v_k, v_1) \geq 0$. ▪

W is cannot be a negative cycle: a contradiction

CONTRADD. AVEVO ASSUNTO FOSSE NEGATIVO

